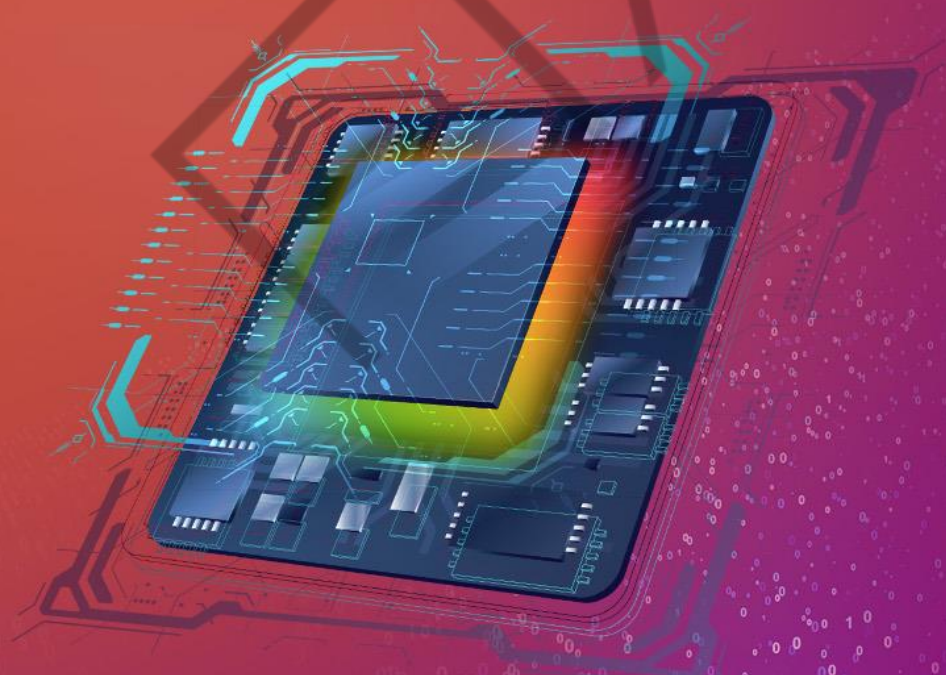


HACKING THE PLANET

GADGETS 10T



9

LISTA DE FIGURAS

Figura 1 – Dispositivos da Hak5 - Kit de Campo	8
Figura 2 – Keylogger anunciado para compra pela Hak5	9
Figura 3 – Laboratório Maker FIAP	10
Figura 4 – Arduino Nano	11
Figura 5 – Hardware Keylogger.....	12
Figura 6 – Localização do Hardware Keylogger na vítima	13
Figura 7 – Caminho percorrido por uma tecla digitada interceptada por um keylogger Wi-Fi.....	14
Figura 8 – Componentes essenciais para o Hardware Keylogger Wi-Fi	15
Figura 9 – Arduino Pro Micro.....	15
Figura 10 – Componentes essenciais para o Hardware Keylogger Wi-Fi	17
Figura 11 – Especificações técnicas do Arduino Pro Micro.....	18
Figura 12 – Conexões de energia do circuito	19
Figura 13 – Protocolos de Comunicação ATmega32u4	20
Figura 14 – Protocolos de Comunicação ESP8266	20
Figura 15 – Protocolos de Comunicação MAX3421E	21
Figura 16 – Jumpers ESP8266 para Arduino Pro Micro.....	22
Figura 17 – Jumpers USB Host Shield para Arduino Pro Micro	23
Figura 18 – Visão final do circuito	23
Figura 19 – Visão frontal com USB Host Shield	24
Figura 20 – Visão traseira com um ATmega32u4 genérico, um Arduino Pro Micro e um ESP8266	24
Figura 21 – Modificação no pino SS ATmega32u4	25
Figura 22 – Página principal do software Arduino IDE	26
Figura 23 – Página inicial do software Arduino IDE	26
Figura 24 – Placas disponíveis para uso no Arduino IDE	27
Figura 25 – Acessando as preferências da Arduino IDE	27
Figura 26 – Adicionando um novo gerenciador de placas à Arduino IDE	28
Figura 27 – Adicionando um novo conjunto de placas à Arduino IDE.....	28
Figura 28 – Adicionando um novo conjunto de placas à Arduino IDE.....	28
Figura 29 – Selecionando uma placa importada externamente	29
Figura 30 – Selecionando a porta de comunicação com o ESP8266.....	29
Figura 31 – Selecionando o código inicial do ESP8266	30
Figura 32 – Código Blink do ESP8266	30
Figura 33 – Verificando a sintaxe do código Blink do ESP8266	31
Figura 34 – LED embutido do ESP8266 aceso	31
Figura 35 – Código Blink ESP8266	32
Figura 36 – Código <i>esp8266_saveSerial</i>	34
Figura 37 – Subindo o código keylogger para o Arduino Leonardo.....	35
Figura 38 – ESP8266 Access Point	36
Figura 39 – BadUSB simulando um teclado.....	36
Figura 40 – Teclas enviadas em tempo real para a página Web, via ESP8266.....	37
Figura 41 – Rubber Ducky.....	38
Figura 42 – Dispositivos providos do microcontrolador ATmega32u4.....	38
Figura 43 – Conectando USB ao ATmega32u4	39
Figura 44 – Attiny85 conectado diretamente à saída USB	39
Figura 45 – ATmega32u4 breakout board.....	40

Figura 46 – Esquemático Arduino Pro Micro	41
Figura 47 – Convertendo DuckyScript para Arduino	42
Figura 48 – Código convertido em Arduino	43
Figura 49 – Selecionando a placa e a porta corretas	43
Figura 50 – Subindo o código para a placa	44
Figura 51 – Resultado do ataque	44
Figura 52 – Conexões do <i>Hardware Keylogger Wi-Fi</i>	46
Figura 53 – Conexões do <i>Hardware Keylogger Wi-Fi</i> usando o Arduino Leonardo ..	46
Figura 54 – Componentes presentes entre a conexão do Arduino Leonardo e o ESP8266	46
Figura 55 – Pinos do ESP-01	47
Figura 56 – Conexões de energia ESP-01	47
Figura 57 – Corrente de operação no ESP8266	48
Figura 58 – Corrente média de saída no pino 3.3V do Arduino Leonardo	48
Figura 59 – Capacitor cerâmico	48
Figura 60 – Sequência de inicialização ESP8266	50
Figura 61 – Circuito com modificações exigidas no datasheet	50
Figura 62 – Conexões de comunicação UART do ESP-01 ao Arduino Leonardo	50
Figura 63 – Dispositivo Masterkey	51
Figura 64 – Página <i>Web Keylogger Wi-Fi Masterkey</i>	52
Figura 65 – Aba de execução de arquivos <i>Duckyscript</i>	52
Figura 66 – Aba de execução <i>DuckyScript</i> em tempo real	53
Figura 67 – Aba de configurações <i>Masterkey</i>	53

LISTA DE QUADROS

Quadro 1 – Componentes essenciais para o <i>Hardware Keylogger Wi-Fi</i>	16
Quadro 2 – Protocolos de Comunicação MAX3421E.....	21
Quadro 3 – Protocolos de Comunicação <i>Hardware Keylogger Wi-Fi</i>	22
Quadro 4 – Funções do código <i>Blink</i>	34
Quadro 5 – Definição de comandos da sintaxe <i>DuckyScript</i>	42
Quadro 6 – Diferença entre tensão e corrente	49

EMANIP

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Primeira parte código <i>Blink</i>	32
Código-fonte 2 – Segunda parte código <i>Blink</i>	33
Código-fonte 3 – Terceira parte código <i>Blink</i>	33
Código-fonte 4 – Código para escrita no notepad com DuckyScript	41

EMANIP

SUMÁRIO

RED TEAM GADGETS	7
1 INTRODUÇÃO	7
2 OS GADGETS.....	8
3 MAKER, MAKER... CADÊ O HACKING?	10
4 HARDWARE KEYLOGGER	12
4.1 Objetivo	13
4.2 Circuito	17
4.3 Código	25
5 BADUSB.....	37
5.1 Objetivo	38
5.2 Circuito	39
5.3 Código	41
6 MASTERKEY	45
6.1 Objetivo	45
6.2 Circuito	45
6.3 Código	51
REFERÊNCIAS.....	54

RED TEAM GADGETS

1 INTRODUÇÃO

O termo *Red Team*, no contexto de cibersegurança, foi herdado das forças armadas estadunidenses, tendo o seu significado bem parecido, apenas mudando o campo de atuação. Dentro das forças armadas, os soldados participam de exercícios de campo para praticar as habilidades, participar de “*war games*” e receber treinamento de novas táticas. Os exercícios de treinamento também podem ocorrer em uma escala muito maior, envolvendo tropas e armamento complexo.

Esses exercícios de treinamento são jogos de guerra com o objetivo de desafiar os processos e práticas operacionais a fim de descobrir fragilidades nesses processos. Inclusive, os jogos de *Capture the Flag* (CTF), como são chamados os jogos de cibersegurança, inicialmente eram chamados de *war games*, remetendo justamente a essa prática militar.

No mundo corporativo de cibersegurança, o *Red Team* é uma área designada a estressar os mecanismos de segurança de forma ofensiva, seja realizando testes de intrusão (*Pentest*) em um escopo específico, emulando adversários com operações simuladas ou fazendo campanhas controladas de malware. Para alcançar esses objetivos, às vezes é necessário ter algumas “cartas na manga”.

Dispositivos que executam comandos na máquina da vítima em uma velocidade supra-humana ou que façam a clonagem de crachás rapidamente, possibilitam uma atuação rápida e precisa, evitando chamar atenção e causar alarmes. Esses dispositivos são *Gadgets* para *Red Teamers* que ao longo deste capítulo aprenderemos a construir!



Figura 1 – Dispositivos da Hak5 - Kit de Campo
Fonte: Hak5 (s.d.)

2 OS GADGETS

Hoje, para adquirir um Red Team Gadget é preciso encontrar uma empresa especializada. Alguns exemplos de lojas que vendem esses tipos de dispositivo:

- Hak5 (<https://shop.hak5.org/collections/all>).
- Hacker House (<https://hackerwarehouse.com/>).
- Lab401 (<https://lab401.com/>).
- Maltronics (<https://maltronics.com/>).
- Keelog (<https://www.keelog.com/>).

Também é possível comprar em lugares como *Aliexpress* e *Bangood*, mas não são todos dispositivos que estarão disponíveis nessas plataformas.

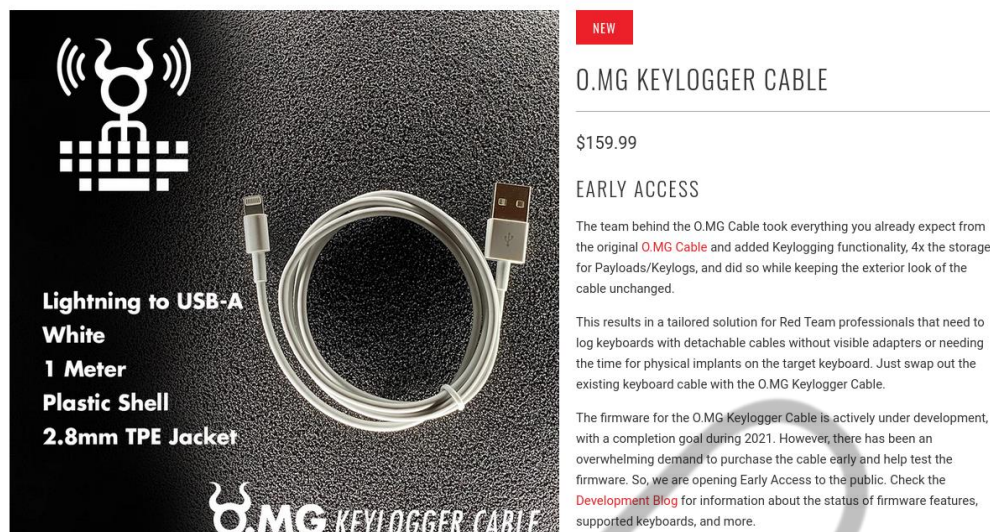


Figura 2 – Keylogger anunciado para compra pela Hak5
Fonte: Elaborada pela autora (2021)

Como é possível notar na Figura “Keylogger anunciado para compra pela Hak5”, dispositivos dessa classe são caros e, por serem vendidos por outros países, demoram bastante para chegar, além de serem taxados na alfândega. Por esse e outros fatores, hoje torna-se difícil e caro adquirir esses equipamentos.

O mundo *Maker*, entretanto, trouxe uma acessibilidade enorme na prototipagem de eletrônica, permitindo até aos iniciantes criarem ferramentas inteligentes. Com o desenvolvimento do Arduino (Hardware e Software), em 2005, iniciou-se uma revolução na área de eletrônica e os primórdios do que entendemos como *Maker* hoje começava a emergir.

A FIAP possui um laboratório *Maker* que permite aos seus alunos um ambiente de uso livre, com impressoras 3D, cortadoras lasers e outras ferramentas de ponta para prototipação. Você pode se informar mais pelo site da FIAP ou entrando em contato com o seu coordenador!

Será usando o Arduino e seus módulos que poderemos criar *Gadgets Red Team* de forma acessível e barata. Neste capítulo, construiremos os seguintes dispositivos, respectivamente:



Figura 3 – Laboratório Maker FIAP
Fonte: Fiap School (2016)

1. *Hardware Keylogger Wi-Fi.*
2. *BadUSB.*
3. *MasterKey.*

Em cada seção serão apresentados em detalhes os componentes usados e links para compra dos dispositivos. Quando possível, serão apresentadas opções para o acompanhamento da seção sem o dispositivo em mãos.

3 MAKER, MAKER... CADÊ O HACKING?

Antes de prosseguirmos, é importante esclarecer alguns pontos: segundo o Dicionário *Michaelis*, “*hacker*” entende-se por:

Indivíduo que se dedica a entender o funcionamento interno de dispositivos, programas e redes de informática com o fim, entre outras coisas, de encontrar falhas em sua segurança ou conseguir um atalho inteligente que possa vir a resultar em um novo recurso ou ferramenta (DICIONÁRIO MICHAELIS ON-LINE, 2021).

Dessa forma, ***hacking*** é o ato de **entender** algo com o fim de explorar caminhos inicialmente desconhecidos. *Hardware hacking* pode ser traduzido, então, como **o ato de entender hardwares com a finalidade de explorar caminhos inicialmente desconhecidos**. Ou seja, para podermos entrar no *hardware hacking*, precisamos, primeiramente, **entender** o *hardware*. Para dizer que entendemos superficialmente o *hardware*, precisamos estar confortáveis em responder extensamente às seguintes perguntas dentro de cada contexto:

1. O que esse hardware faz?
2. Quais tecnologias ele usa?
3. Que arquitetura é usada?
4. Como é feito o processamento de informação?

Uma maneira divertida e eficiente de dominar um assunto é fazendo você mesmo! Por isso, é importante compreender que a construção dos dispositivos de *Red Team* neste curso tem o intuito de facilitar o entendimento de *hardware* em si, uma vez que serão feitos a programação de microcontroladores, o estudo de protocolos de comunicação e a análise de componentes eletrônicos, mas não pode ser considerado cegamente como *Hardware Hacking*.

No mundo de Segurança da Informação, esses dispositivos podem ser úteis, não só por serem facilitadores em casos de invasão física, mas também porque trazem conceitos complexos de exploração! Essa é uma maneira útil, divertida e pertinente para auxiliar no entendimento de *hardware*, sem perder o foco no objetivo: *hacking*.

Antes de destruir, vamos construir um pouco?

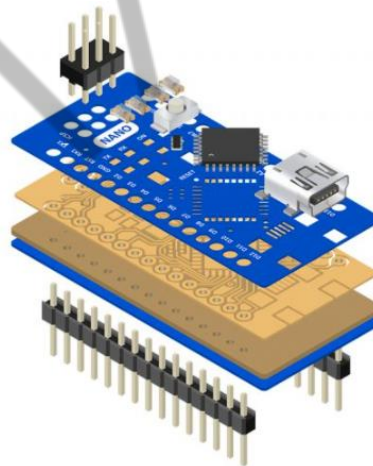


Figura 4 – Arduino Nano
Fonte: Shutterstock (2021)

4 HARDWARE KEYLOGGER

Um *keylogger*, genericamente falando, é um dispositivo ou código que passivamente monitora e grava teclas digitadas em um computador. Como será feito o implante do *keylogger* e como o atacante acessará as teclas que foram digitadas é o que permite o *keylogging* ser um assunto tão extenso.

Estudos sobre *keylogging* vem sendo conduzidos há anos e podem ser incorporados nas várias camadas de abstração de um computador, seja entre o teclado e a porta USB do computador, conhecidos como “*Hardware Keyloggers*”, ou diretamente no Sistema Operacional, conhecidos como “*Software Keyloggers*”.



Figura 5 – Hardware Keylogger
Fonte: Google Imagens (2017)

Para quebrarmos o gelo, vamos começar construindo um simples *hardware keylogger* que envia as teclas via Wi-Fi. Os seguintes materiais serão utilizados:

- ATmega32u4 (Pode ser qualquer Arduino que possua esse microcontrolador).
 - Pro Micro
 - <https://www.filipeflop.com/produto/placa-leonardo-pro-micro-5v/>
 - Leonardo (RECOMENDADO)
 - <https://www.filipeflop.com/produto/placa-leonardo-r3-cabo-usb-para-arduino/>
- ESP8266

- <https://www.filipeflop.com/produto/modulo-wifi-esp8266-nodemcu-esp-12/>
- USB Host Shield
 - <https://www.filipeflop.com/produto/usb-host-shield-android-para-arduino/>

4.1 Objetivo

Pensando em onde queremos chegar: **capturar teclas digitadas por uma vítima no computador e transmiti-las via Wi-Fi**, qual caminho lógico temos de percorrer? Vamos pensar juntos.

Já sabemos que vamos criar um *hardware keylogger*, portanto precisamos de um dispositivo que fique **entre** o teclado USB e a porta USB do computador da vítima.

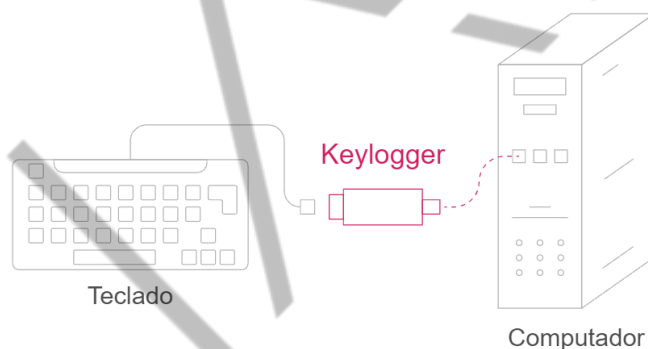


Figura 6 – Localização do Hardware Keylogger na vítima
Fonte: Elaborada pela autora (2021)

Dessa forma, todas as teclas digitadas pelo teclado USB passarão pelo *hardware keylogger* antes de alcançarem o computador. Agora, como faremos para enviar essas teclas digitadas para o atacante?

Existem diversas maneiras, cada uma com suas limitações e facilidades. Uma forma bem simples é salvar as teclas no próprio *keylogger* com uma memória adicional, um cartão *micro SD*, por exemplo, entretanto isso depende de o atacante pegar (fisicamente) o *keylogger* implantado para acessar os *logs*, portanto essa forma não é muito prática! Também é possível enviar as teclas via Bluetooth, mas um módulo Bluetooth, como o HC-05, consegue enviar dados a uma taxa muito menor que o

ESP8266 (Wi-Fi) e isso é um impedimento quando gostaríamos de receber as teclas digitadas em tempo real. Como determinado antes, nós escolhemos enviar via Wi-Fi.

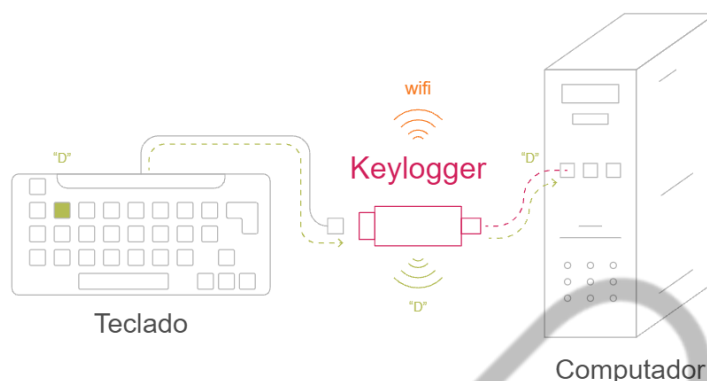


Figura 7 – Caminho percorrido por uma tecla digitada interceptada por um keylogger Wi-Fi
Fonte: Elaborada pela autora (2021)

Note na Figura “Caminho percorrido por uma tecla digitada interceptada por um keylogger Wi-Fi” o caminho, em verde, que uma tecla digitada pela vítima deve percorrer. Assim que ela alcançar o *keylogger*, é preciso tanto enviá-la via Wi-Fi para o atacante quanto manter o tráfego inicial e transferi-la para o computador. Esse processo é difícil de ser calibrado e muitas vezes resulta em um mapeamento errado do teclado, fazendo com que o “ç” digitado chegue como “&” no computador da vítima. Vamos entender melhor por que isso acontece mais adiante!

Agora que já entendemos o nosso ataque em alto nível, vamos focar na camada eletrônica. Novamente, existem diversas maneiras de arranjar conjuntos de componentes com um objetivo particular. Podemos criar esse *keylogger* programando diretamente circuitos lógicos, usando tecnologias CPLD (*Complex Programmable Logic Device*) ou FPGA (*Field Programmable Gate Array*), podemos construir a antena Wi-Fi ou fazer um circuito inteiramente do zero, com uma arquitetura única, jamais vista. Todas essas opções são válidas, mas exigem esforços diferentes. Iremos optar pelo caminho mais fácil, usando um hardware já inicialmente projetado e modular: Arduino.

Vamos descer o nível e pensar logicamente. Sem subestimar nenhuma etapa, como fazemos com que a tecla digitada seja acessada via Wi-Fi em tempo real, ao mesmo tempo que transmite para o computador?

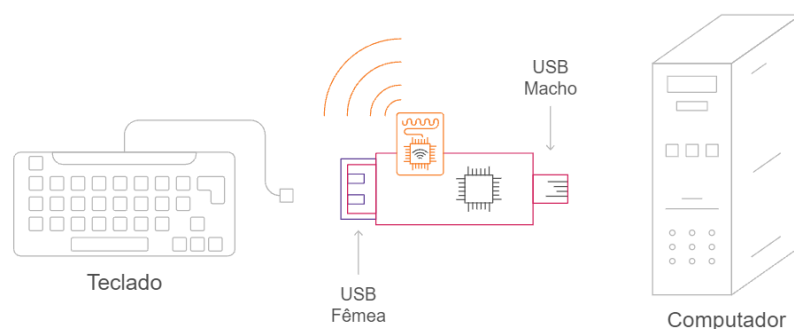


Figura 8 – Componentes essenciais para o Hardware Keylogger Wi-Fi
Fonte: Elaborada pela autora (2021)

A primeira etapa a ser feita é entender o que o nosso Arduino pode e não pode fazer. Eu estou usando para este projeto o Arduino Pro Micro. Ao pesquisar na página oficial do Arduino (<https://www.arduino.cc/>), recebemos algumas informações interessantes:

8 bit AVR < 20 mA Microusb 5V Standard (~20) No battery

O Arduino Pro Micro vem com um USB embutido que o torna reconhecível como um mouse ou teclado.

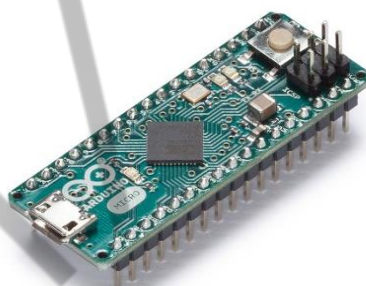


Figura 9 – Arduino Pro Micro
Fonte: Elaborada pela autora (2021)

Como vemos na Figura “Componentes essenciais para o Hardware Keylogger Wi-Fi”, precisamos que o nosso dispositivo tenha uma entrada e uma saída USB para que o teclado seja inserido em um lado, enquanto o *keylogger* encaixa na entrada USB do computador. Precisamos, também, que esse dispositivo tenha a capacidade de transmitir via Wi-Fi.

Na Figura “Arduino Pro Micro”, é possível perceber que esse Arduino possui a capacidade embutida de ser reconhecido como um teclado pelo computador, além de apresentar uma entrada micro USB que, com um adaptador, pode ser facilmente convertida para uma saída USB. Se continuarmos lendo a página, não é feita

nenhuma menção a Wi-Fi, o que dá a entender que esse dispositivo não possui essa capacidade nativamente e teremos de incluí-la externamente.

Ou seja, externamente ao Arduino é preciso adquirir um módulo Wi-Fi e um adaptador USB fêmea. Para o módulo Wi-Fi, o ESP8266 é um componente muito comum e barato e que atende as necessidades, mas e o adaptador USB fêmea? Como receber os dados do teclado no Arduino?

O padrão USB exige a presença de um *host* e um *client* para realizar a comunicação. Quando um teclado é inserido no computador, o computador se comporta como *host* USB, enquanto o teclado age como *client* USB. Sabendo que a classe USB para teclados é *Human Interface Device* (HID), podemos dizer com mais precisão que o teclado age como um *client* USB HID. Concluímos, assim, que o Arduino precisa atuar como *host* e *client* HID dessa comunicação USB.

Se voltarmos à Figura “Arduino Pro Micro”, vemos que o Arduino Pro Micro possui nativamente a função de *client* USB HID, como afirmado em “vem com um USB embutido que o torna reconhecível como um mouse ou teclado”. Ser “reconhecível” implica o Arduino ser o *client* da comunicação e “como um mouse ou teclado” determina que ele pode ser apresentado ao *host* USB como um dispositivo USB de classe HID.

Bom, sabemos que o Arduino Pro Micro possui nativamente a capacidade de atuar como um *client* USB HID, mas não vemos nenhuma menção a atuar como *host* USB, o que significa que teremos de importar essa capacidade de forma externa, assim como o módulo Wi-Fi. Uma busca rápida no Google por módulos *host* USB para Arduino, chegamos a alguns que realizam essa função, a maioria utilizando um mesmo Circuito Integrado (CI) de nome “MAX3421E”.

Nossa, quanta informação! Se juntarmos tudo em uma tabela, talvez possamos simplificar um pouco a ideia:

Capacidade	Arduino possui nativamente?	Módulo externo
USB host (USB fêmea)	Não	Sim: MAX3421E
USB client HID (USB macho)	Sim	-
Wi-Fi	Não	Sim: ESP8266

Quadro 1 – Componentes essenciais para o *Hardware Keylogger Wi-Fi*
Fonte: Elaborado pela autora (2021)

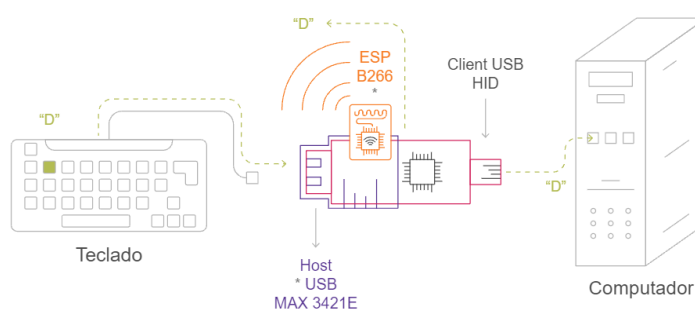


Figura 10 – Componentes essenciais para o Hardware Keylogger Wi-Fi
Fonte: Elaborada pela autora (2021)

A Figura “Componentes essenciais para o Hardware Keylogger Wi-Fi” mostra os requisitos finais para o desenvolvimento do *Hardware Keylogger Wi-Fi* que conseguimos seguindo um caminho lógico e sem a necessidade de um conhecimento profundo no assunto.

A estrutura de análise que foi usada até aqui é, também, a lógica para a exploração de *hardwares*. Muitos dispositivos embarcados proprietários aparecem sem informação alguma sobre o circuito; os chips são raspados para esconder o código do fabricante e são inseridas várias vias “dummy”, ou seja, vias que não estão conectadas a nada e a sua presença é para meramente confundir o observador.

Essas técnicas de ofuscação da placa podem trazer bastante dor de cabeça na hora de realizar a Engenharia Reversa de circuitos, mas são metodologias de análise como as que usamos até aqui que auxiliam profundamente a reverter esses impedimentos.

4.2 Circuito

Na Idade Média, eram chamados de magos e alquimistas, os cientistas que se utilizavam de recursos físicos para fazer “magia”. Nessa época, a manipulação química de elementos era vista como sobrenatural e poucos sabiam como era feita. A possibilidade de receber as teclas digitadas no computador de outra pessoa apenas inserindo um adaptador USB pode parecer para alguns como “magia”, mas, na verdade, estamos manipulando a eletricidade a nosso favor.

Para o desenvolvimento de nosso *Keylogger*, é preciso ter conhecimento inicialmente das especificações elétricas de cada módulo a ser usado. Esses detalhes

podem ser consultados no *datasheet* ou no site do fornecedor de cada componente/módulo.

Se voltarmos ao site oficial do Arduino Pro Micro (<https://store-usa.arduino.cc/products/arduino-micro?selectedStore=us>), vemos que dispositivo opera a uma tensão de 5V, mas pode receber de 7 a 12V de entrada.

Microcontroller	ATmega32U4
Operating Voltage	5V
Input Voltage (recommended)	7-12V
Digital I/O Pins	20
PWM Channels	7
Analog Input Channels	12
DC Current per I/O Pin	20 mA
DC Current for 3.3V Pin	50 mA
Flash Memory	32 KB (ATmega32U4) of which 4 KB used by bootloader
SRAM	2.5 KB (ATmega32U4)
EEPROM	1 KB (ATmega32U4)

Figura 11 – Especificações técnicas do Arduino Pro Micro
Fonte: ARDUINO (2023)

A placa pode operar com alimentação externa de 7 a 20 volts. Se for fornecido com menos de 7 V, entretanto, o pino de 5 V pode fornecer menos de cinco volts e a placa pode ficar instável. Se usar mais de 12V, o regulador de tensão pode superaquecer e danificar a placa. A faixa recomendada é de 7 a 12 volts.

Pesquisando as especificações técnicas dos outros módulos, entendemos que o USB Host Shield opera a 5V, enquanto o ESP8266 opera entre 2.5V e 3.6V. Como “Magos da Eletricidade”, como podemos fornecer essas voltagens de operação?

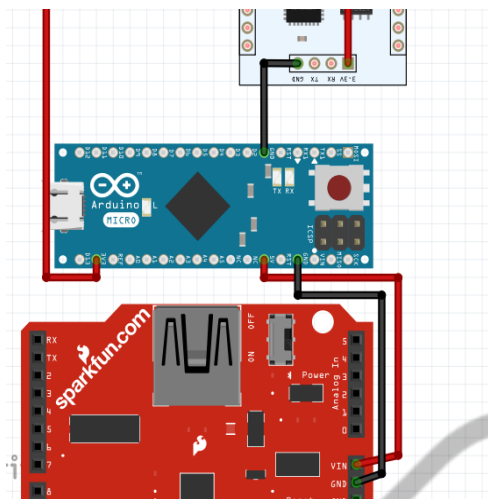


Figura 12 – Conexões de energia do circuito
Fonte: Elaborada pela autora (2021)

Uma forma de fazer isso é alimentar o Arduino pela entrada micro USB, pois a tensão de operação dele é igual ou maior (5V) que a dos outros módulos (5V USB Host Shield e 3.6V ESP8266), e fornecer a energia de saída do próprio Arduino para os outros módulos, como mostrado na Figura “Conexões de energia do circuito”. Dessa forma, estamos permitindo que todos os componentes estejam recebendo energia e, portanto, estejam operacionais! Mas, como já é de imaginar, isso não é suficiente para sairmos roubando teclas por aí.

É necessário que os módulos possam se comunicar para trocarem informações do nosso programa. Cada módulo tem seu papel a desempenhar e precisa reportar os resultados e solicitar informações com o “cérebro” do circuito (Arduino), em que é feito o processamento central. Para essa comunicação ser possível, existem algumas “línguas” que os componentes podem “falar”, denominadas de “Interfaces de Comunicação”.

Cada interface possui o seu protocolo, com suas regras e, assim, como as pessoas podem falar mais de uma língua, componentes podem ter a capacidade de se comunicar por diversas interfaces. Veremos com mais detalhes ao longo do curso, mas alguns protocolos comuns de comunicação de hardware são:

- UART (*Universal Asynchronous Receiver/Transmitter*).
- SPI (*Serial Peripheral Interface*).
- I²C (*Inter-Integrated Circuit*).

Como faremos a troca de informações entre os módulos, precisamos entender qual “língua” cada módulo sabe “falar”. E como podemos obter essa informação, você sabe dizer?

Novamente, o melhor lugar para encontrarmos qualquer informação sobre um componente é em seu *datasheet*. O *datasheet* normalmente vem em formato PDF e traz informações técnicas e de desempenho sobre um produto específico. Quando não encontramos o *datasheet* de um dispositivo completo, como é o caso do Arduino Pro Micro e USB Host Shield, podemos procurar referenciando o componente principal de cada ativo, no caso ATmega32u4 e MAX3421E respectivamente.

Vejamos os protocolos de comunicação de cada módulo nas imagens abaixo:

- Two 16-bit Timer/Counter with Separate Prescaler, Compare- and Capture Mode
- One 10-bit High-Speed Timer/Counter with PLL (64MHz) and Compare Mode
- Four 8-bit PWM Channels
- Four PWM Channels with Programmable Resolution from 2 to 16 Bits
- Six PWM Channels for High Speed Operation, with Programmable Resolution from 2 to 11 Bits
- Output Compare Modulator
- 12-channels, 10-bit ADC (features Differential Channels with Programmable Gain)
- Programmable Serial USART with Hardware Flow Control
- Master/Slave SPI Serial Interface
- Byte Oriented 2-wire Serial Interface
- Programmable Watchdog Timer with Separate On-chip Oscillator
- On-chip Analog Comparator
- Interrupt and Wake-up on Pin Change
- On-chip Temperature Sensor
- Special Microcontroller Features

Figura 13 – Protocolos de Comunicação ATmega32u4

Fonte: Elaborada pela autora (2021)

Hardware	CPU	Tensilica L106 32-bit processor
	Peripheral Interface	UART/SDIO/SPI/I2C/I2S/IR Remote Control
		GPIO/ADC/PWM/LED Light & Button
	Operating Voltage	2.5 V ~ 3.6 V
	Operating Current	Average value: 80 mA
	Operating Temperature Range	-40 °C ~ 125 °C
	Package Size	QFN32-pin (5 mm x 5 mm)

Figura 14 – Protocolos de Comunicação ESP8266

Fonte: Elaborada pela autora (2021)



USB Peripheral/Host Controller with **SPI Interface**

Description

The controller contains all necessary logic to implement a full-/low-speed USB 2.0. A protection and connect. An internal low-level USB and bus retries. or set accessed to 26MHz. Any 2, etc.) can add the simple 3-

selection of USB processor, ASIC, or

Features

- ◆ Microprocessor-Independent USB Solution
- ◆ Software Compatible with the MAX3420E USB Peripheral Controller with SPI Interface
- ◆ Complies with USB Specification Revision 2.0 (Full-Speed 12Mbps Peripheral, Full-/Low-Speed 12Mbps/1.5Mbps Host)
- ◆ Integrated USB Transceiver
- ◆ Firmware/Hardware Control of an Internal D+ Pullup Resistor (Peripheral Mode) and D+/D- Pulldown Resistors (Host Mode)
- ◆ **Programmable 3- or 4-Wire, 26MHz SPI Interface**
- ◆ Level Translators and V_L Input Allow Independent System Interface Voltage

Figura 15 – Protocolos de Comunicação MAX3421E
Fonte: Elaborada pela autora (2021)

Juntamente às especificações de energia, temos:

Dispositivo	Energia	Comunicação
Arduino Pro Micro (ATmega32u4)	5V	SPI/USART
USB Host Shield (MAX3421E)	5V	SPI
ESP8266	2.5 - 3.6V	SPI/UART

Quadro 2 – Protocolos de Comunicação MAX3421E
Fonte: Elaborado pela autora (2021)

Uma forma de criarmos uma comunicação entre o Arduino e seus módulos é colocarmos o Arduino para se comunicar via SPI com o *USB Host Shield* e via UART com o ESP8266.

Todo protocolo, seja de hardware ou não, terá seus padrões e definições próprios. No caso de hardware, temos isso também refletido em portas físicas do dispositivo, como “pinos”.

Neste capítulo não aprofundaremos nos detalhes dos protocolos de comunicação, mas, sim, nos pinos e conexões físicas desses protocolos. Cada dispositivo terá seu mapeamento próprio desses pinos, podendo ser consultado por meio de cada *datasheet*. Para facilitar, vou expandir a tabela acima com as informações sobre os pinos de comunicação que extraí dos *datasheets*.

Dispositivo	Energia	Comunicação	Pinout
Arduino Pro Micro	5V	SPI	15 SCK
			14 MISO
			16 MOSI
			10 SS
		UART	1 TX
			2 RX
USB Host Shield	5V	SPI	13 SCK
			12 MISO
			11 MISO
			10 SS
ESP8266	2.5V - 3.6V	UART	2 TX
			7 RX

Quadro 3 – Protocolos de Comunicação *Hardware Keylogger Wi-Fi*

Fonte: Elaborado pela autora (2021)

Conecte cada pino correspondente. Para UART, devemos inverter os pinos (entenderemos melhor a razão no próximo capítulo), por ora, siga o esquema abaixo.

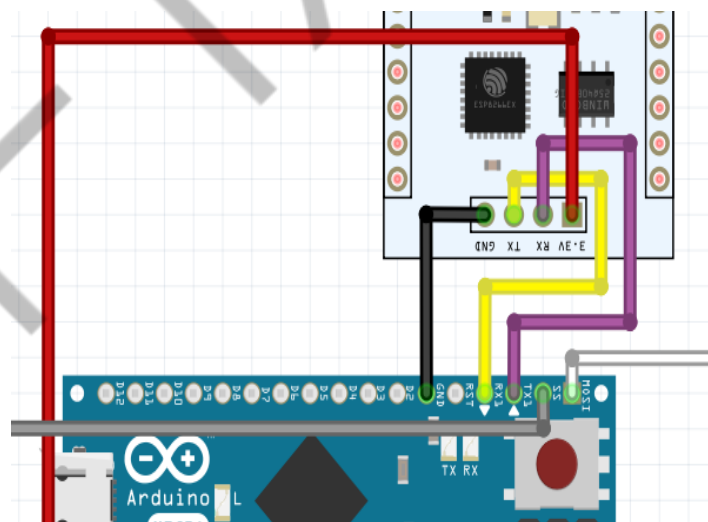


Figura 16 – Jumpers ESP8266 para Arduino Pro Micro

Fonte: Elaborada pela autora (2021)

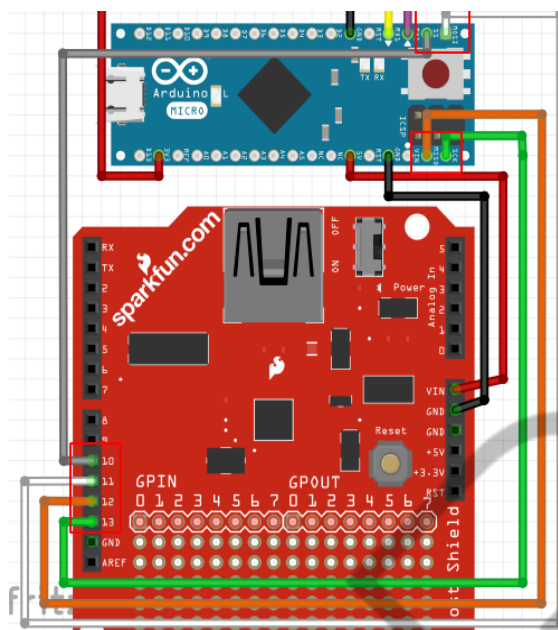


Figura 17 – Jumpers USB Host Shield para Arduino Pro Micro
Fonte: Elaborada pela autora (2021)

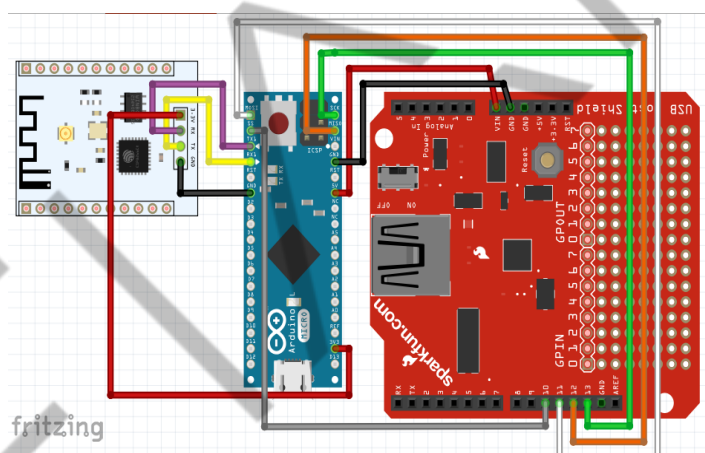


Figura 18 – Visão final do circuito
Fonte: Elaborada pela autora (2021)

Dependendo dos dispositivos que você adquiriu, a sua visão final do circuito deve estar diferente da colocada na Figura “Visão final do circuito”. A minha, por exemplo, está assim nesse momento:

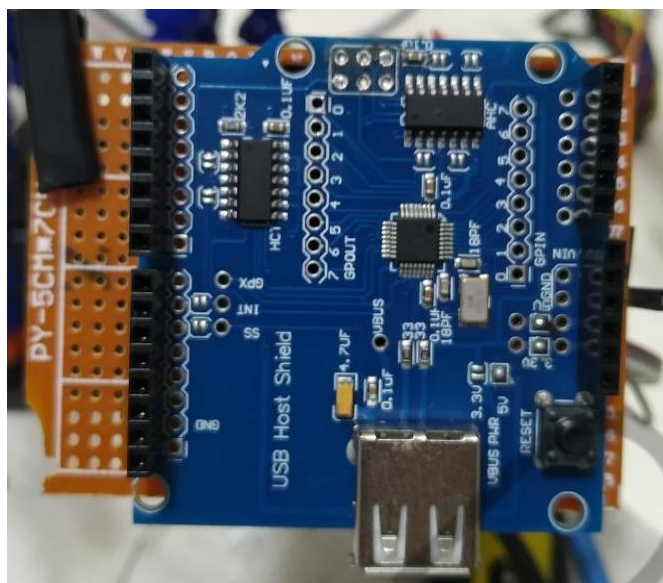


Figura 19 – Visão frontal com USB Host Shield
Fonte: Elaborada pela autora (2021)

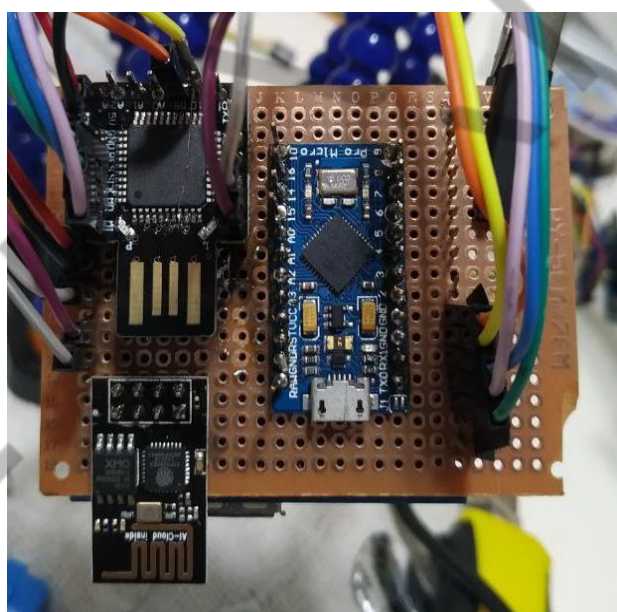


Figura 20 – Visão traseira com um ATmega32u4 genérico, um Arduino Pro Micro e um ESP8266
Fonte: Elaborada pela autora (2021)

Não se assuste com o meu *Frankenstein*! Às vezes quando vamos trabalhar com o que já possuímos no nosso laboratório encontramos dispositivos com mau funcionamento e precisamos adaptar.

No meu caso, alguns pinos dos meus dispositivos ATmega32u4 não estavam respondendo e eu precisei improvisar. Inclusive, durante a prototipação, notei que o pino SS para comunicação SPI do meu ATmega32u4 genérico não possuía uma saída em um pino macho e eu tive de “hackear” o circuito para fazê-lo funcionar, como mostra a Figura “Modificação no pino SS ATmega32u4”.

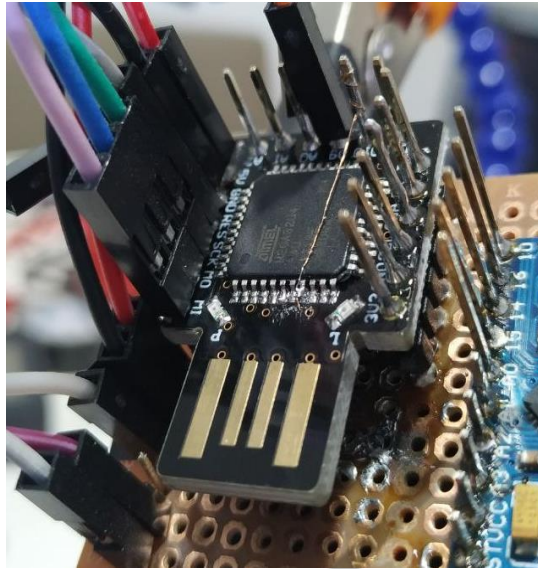


Figura 21 – Modificação no pino SS ATmega32u4
Fonte: Elaborada pela autora (2021)

Diferentemente do que muitos pensam, eletrônica não é algo fixo ou imutável, por mais que um circuito se mostre de uma maneira, podemos sempre realizar modificações a nosso favor. O que precisei fazer, mesmo que sem o intuito malicioso, é uma forma de *Hardware Hacking* e, como veremos, modificar um pino em um circuito pode ser a peça-chave para desenrolar os segredos que ele guarda.

4.3 Código

1) Com o circuito em mãos podemos seguir para o código. O Arduino permite ser programado usando uma IDE (*Integrated Development Environment*) própria. Siga para o site <<https://www.arduino.cc/en/software>> e baixe a IDE compatível com seu Sistema Operacional.

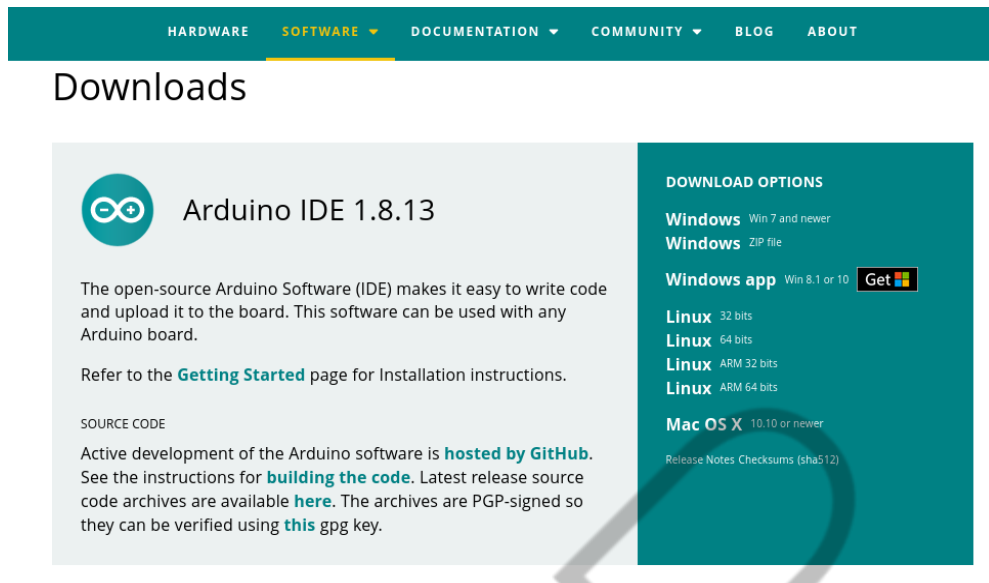


Figura 22 – Página principal do software Arduino IDE
Fonte: Elaborada pela autora (2021)

2) Uma vez feito o download, abra-a. Você deve estar se deparando com uma tela como a da figura a seguir.

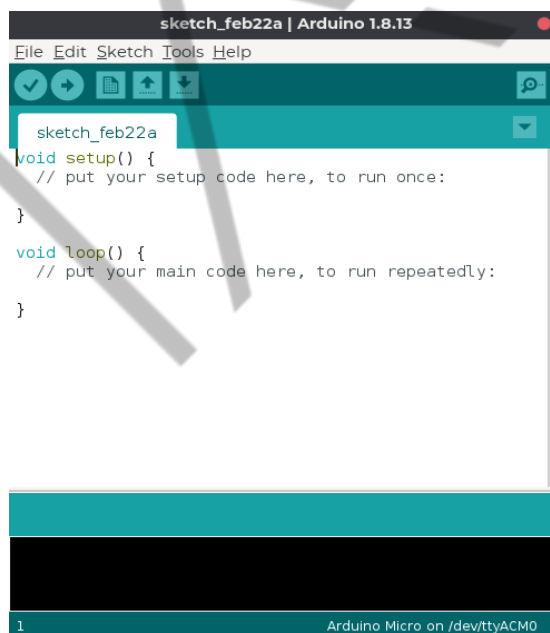


Figura 23 – Página inicial do software Arduino IDE
Fonte: Elaborada pela autora (2021)

Toda vez que mudarmos de placa, devemos verificar antes se a IDE possui o dispositivo dentro de "Tools" > "Board: ".

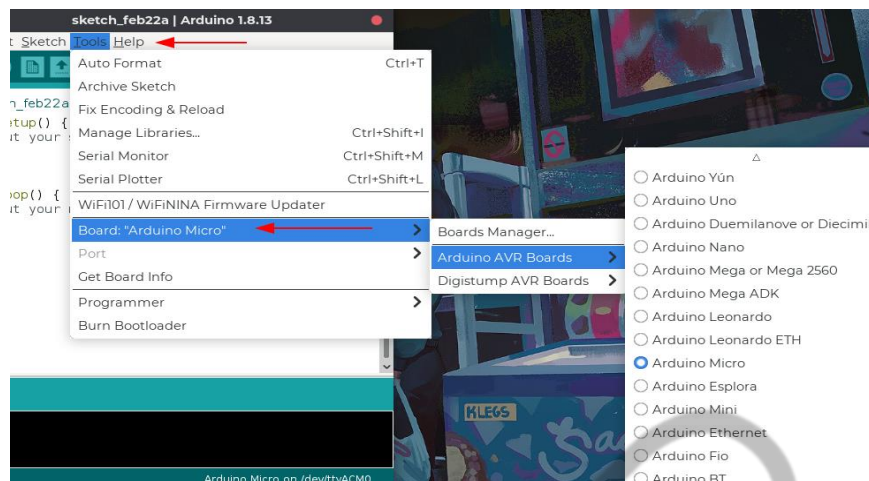


Figura 24 – Placas disponíveis para uso no Arduino IDE
Fonte: Elaborada pela autora (2021)

3) Para o *Hardware Keylogger Wi-Fi*, precisaremos programar dois dispositivos: Arduino Leonardo ou Pro Micro e ESP8266. Podemos ver pela Figura “Placas disponíveis para uso na Arduino IDE” que o Arduino Pro Micro e Leonardo já estão disponíveis, mas o ESP8266 não. Para isso, temos de importá-lo para a IDE.

Seguindo as instruções do site oficial do Arduino, <<https://projecthub.arduino.cc/PatelDarshil/getting-started-with-nodemcu-esp8266-on-arduino-ide-b193c3>> devemos começar adicionando a seguinte URL em “File” > “Preferences” > “Additional Board Manager URLs” > “Ok”.

http://arduino.esp8266.com/stable/package_esp8266com_index.json

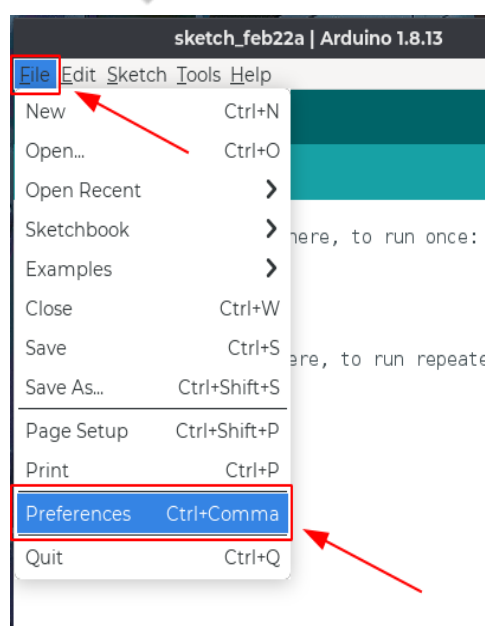


Figura 25 – Acessando as preferências da Arduino IDE
Fonte: Elaborada pela autora (2021)

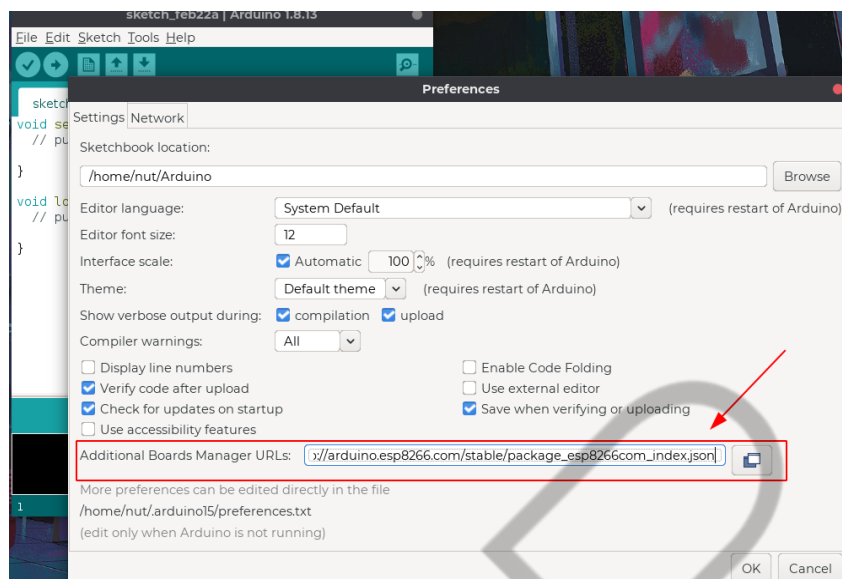


Figura 26 – Adicionando um novo gerenciador de placas à Arduino IDE
Fonte: Elaborada pela autora (2021)

Agora que adicionamos um gerenciador de placas externo, devemos importá-lo para ser consumido em nossa IDE. Procure a palavra “ESP8266” em “Tools” > “Board:” > “Boards Manager...”.



Figura 27 – Adicionando um novo conjunto de placas à Arduino IDE
Fonte: Elaborada pela autora (2021)

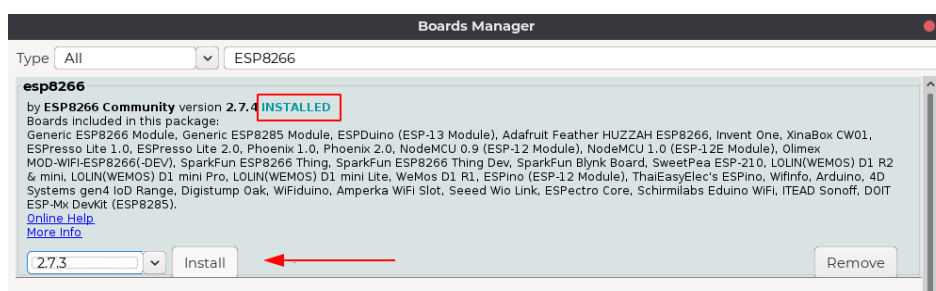


Figura 28 – Adicionando um novo conjunto de placas à Arduino IDE
Fonte: Elaborada pela autora (2021)

Uma vez instalado, você deve perceber que na mesma aba da Figura “Placas disponíveis para uso na Arduino IDE” aparecem novas placas disponíveis.

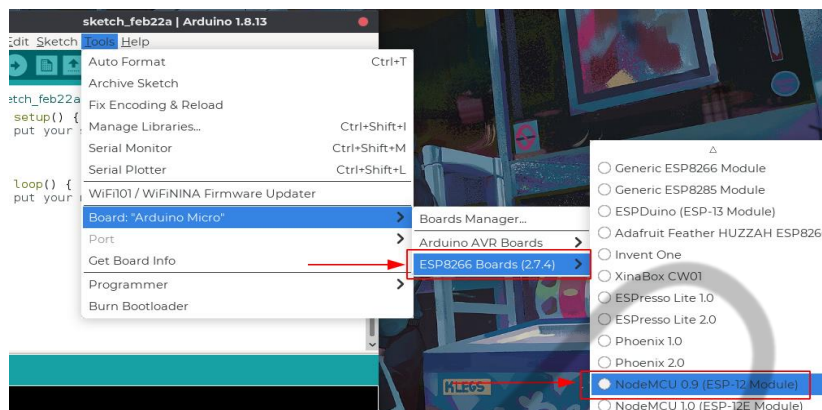


Figura 29 – Selecionando uma placa importada externamente
Fonte: Elaborada pela autora (2021)

Selecione a placa correspondente ao seu dispositivo. Se você adquiriu pelo link sugerido, então a sua placa será “NodeMCU 0.9 (ESP-12 Module)”. Conecte o ESP8266 ao computador e desconecte temporariamente os pinos TX e RX da placa. Isso é necessário para não ter a interferência serial, visto que é dessa forma que ele se comunica, também, com o seu computador.

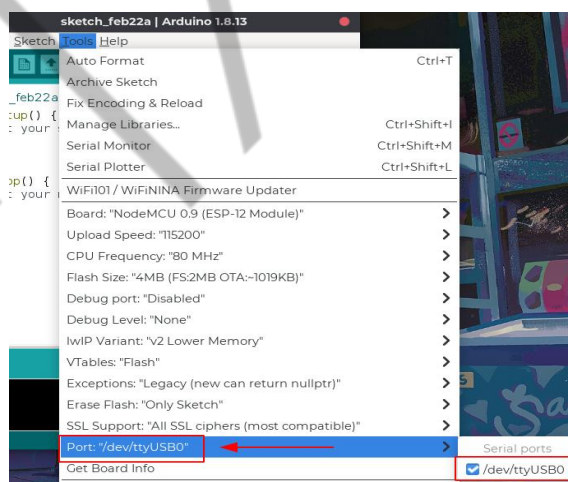


Figura 30 – Selecionando a porta de comunicação com o ESP8266
Fonte: Elaborada pela autora (2021)

Caso você esteja usando a IDE por uma distro Linux, sua porta será parecida com a da Figura “Selecionando a porta de comunicação com o ESP8266”, “/dev/ttyUSB”, mas caso esteja usando MacOS, verá algo como “/dev/cu.” ou “/dev/tty.”; ou Windows, deve ver “COM1” ou “COM” seguido de um número. Selecione a porta correspondente ao seu dispositivo e pronto! Seu ambiente de desenvolvimento ESP8266 está preparado!

Para ter certeza de que tudo está funcionando corretamente antes de prosseguir, é uma boa prática rodar um código “Hello World” em Arduino. Esse código normalmente é chamado de “Blink” para placas de desenvolvimento, pois faz o led embutido à placa brilhar com um delay. Vá em “File” > “Examples” > “ESP8266” > “Blink”, como na Figura “Selecionando o código inicial do ESP8266”.

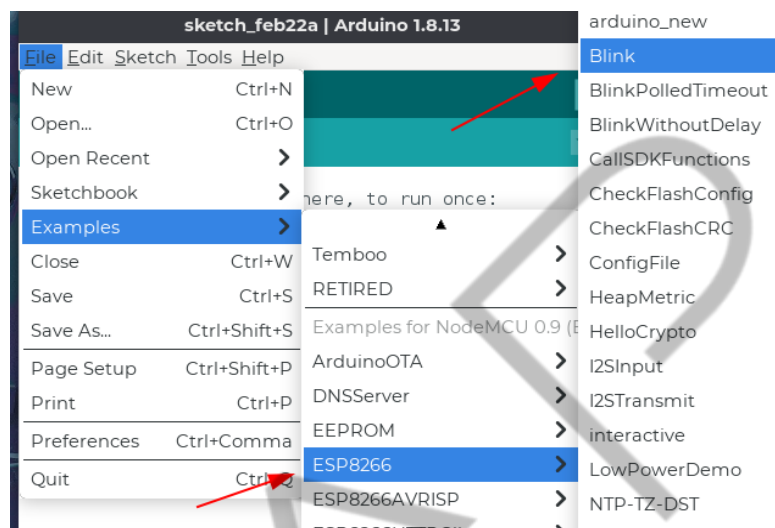


Figura 31 – Selecionando o código inicial do ESP8266
Fonte: Elaborada pela autora (2021)

Ao clicar no item, deve ter sido aberta uma nova janela com o código.

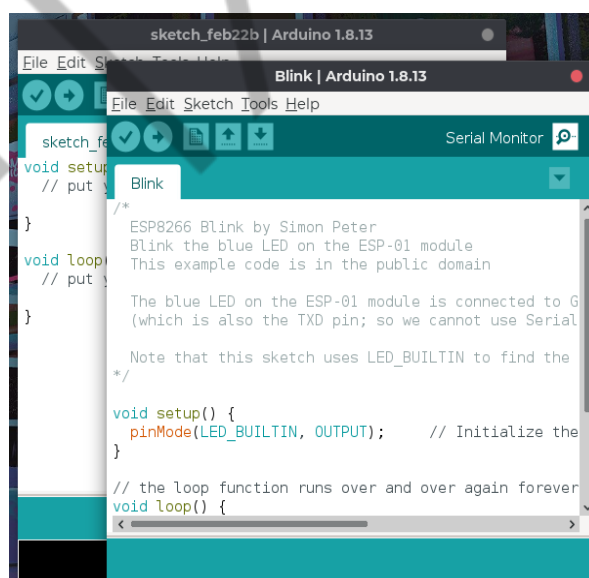


Figura 32 – Código Blink do ESP8266
Fonte: Elaborada pela autora (2021)

Sem realizar nenhuma modificação, clique no botão “Verify” para checarmos por erros de sintaxe no código. Se você recebeu a mensagem “Done Compiling”, o código não apresentou erros!

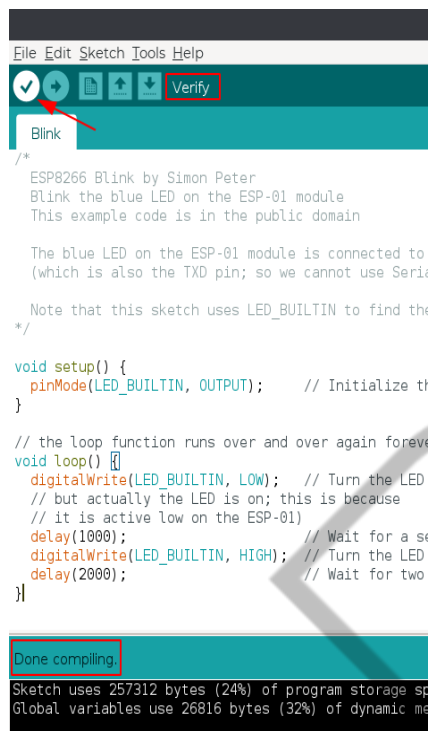


Figura 33 – Verificando a sintaxe do código Blink do ESP8266
Fonte: Elaborada pela autora (2021)

Cheque na aba “Tools” se as configurações da Figura “Selecionando a porta de comunicação com o ESP8266” continuam as mesmas, clique em “Upload” e aguarde até receber a mensagem “Done Compiling”. Se tudo foi como planejado, você deve estar vendo um LED pequeno acendendo e apagando com certos intervalos!

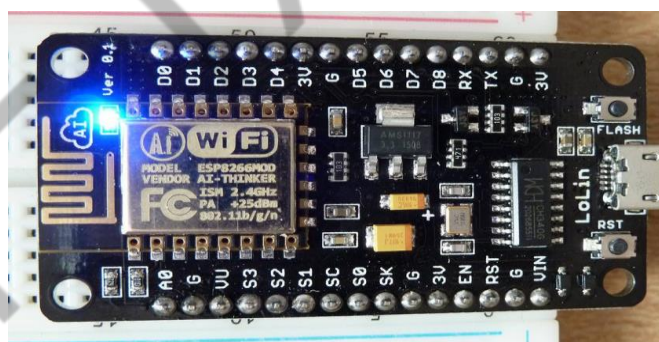


Figura 34 – LED embutido do ESP8266 aceso
Fonte: Elaborada pela autora (2021)

Parabéns! Você agora está interagindo com o *Hardware* via *Software*! Pare um momento e analise a estrutura do código *Blink*. Você consegue apontar em quais linhas de código é feita a definição dos intervalos de luminosidade? E onde é declarado que o LED deve ser aceso?

Vamos entender, linha a linha, o que esse código está fazendo. Ele serve como base para qualquer outro código que será escrito na estrutura APL (*Arduino Programming Language*).

```
/*
  ESP8266 Blink by Simon Peter
  Blink the blue LED on the ESP-01 module
  This example code is in the public domain

  The blue LED on the ESP-01 module is connected to GPIO1
  (which is also the TXD pin; so we cannot use Serial.print() at the same time)

  Note that this sketch uses LED_BUILTIN to find the pin with the internal LED
  */

void setup() {
  pinMode(LED_BUILTIN, OUTPUT); // Initialize the LED_BUILTIN pin as an output
}

// the loop function runs over and over again forever
void loop() {
  digitalWrite(LED_BUILTIN, LOW); // Turn the LED on (Note that LOW is the voltage level
  // but actually the LED is on; this is because
  // it is active low on the ESP-01)
  delay(1000); // Wait for a second
  digitalWrite(LED_BUILTIN, HIGH); // Turn the LED off by making the voltage HIGH
  delay(2000); // Wait for two seconds (to demonstrate the active low LED)
}
```

Figura 35 – Código Blink ESP8266
Fonte: Elaborada pela autora (2021)

Em verde, temos a primeira parte do código *Blink*. Essa parte, por estar entre “/*” e “*/” notamos que se trata de um comentário. Os comentários são importantes porque trazem informações relevantes do código, como notamos quando Simon Peter comenta que o led azul do ESP8266 está conectado ao *GPIO1* e que este, por sua vez, também se trata do pino TX! Ou seja, se chamarmos uma comunicação serial enquanto interagimos com o LED azul, provavelmente não obteremos o resultado esperado. Ele também explica que o código usa de uma função nativa do Arduino, chamada *LED_BUILTIN*, que já procura pelo pino relativo ao LED interno sem precisarmos declará-lo manualmente.

```
/*
  ESP8266 Blink by Simon Peter
  Blink the blue LED on the ESP-01 module
  This example code is in the public domain

  The blue LED on the ESP-01 module is connected to GPIO1
  (which is also the TXD pin; so we cannot use Serial.print() at the same time)

  Note that this sketch uses LED_BUILTIN to find the pin with the internal LED
  */
```

Código-fonte 1 – Primeira parte código *Blink*
Fonte: Elaborado pela autora (2021)

Em azul, temos a segunda parte do código *Blink*. Perceba que é feita a chamada de uma função nativa da linguagem Arduino denominada “pinMode()”. Segundo a documentação oficial, entendemos que essa função permite definir se um pino será de entrada ou saída. No caso, é declarado que a variável *LED_BUILTIN* será um pino de saída, pois deverá mandar energia para o LED azul da placa.

```
void setup() {  
  pinMode(LED_BUILTIN, OUTPUT);    //pinMode(pino, modo)  
}
```

Código-fonte 2 – Segunda parte código *Blink*
Fonte: Elaborado pela autora (2021)

Temos a terceira parte do código *Blink*. Aqui são usadas duas funções nativas do Arduino, “digitalWrite()” e “delay()”. Seguindo a mesma documentação oficial citada acima, entendemos que a função “digitalWrite()” manda um sinal digital que pode ser alto (5V ou 3.3V) ou baixo (0V) e que a função “delay()” configura um atraso, em milissegundos, na execução do programa.

```
void loop() {  
  digitalWrite(LED_BUILTIN, LOW); // Manda um sinal baixo para o pino  
  delay(1000);                    // Espera um segundo  
  digitalWrite(LED_BUILTIN, HIGH); // Manda um sinal alto para o pino  
  delay(2000);                    // Espera dois segundos  
}
```

Código-fonte 3 – Terceira parte código *Blink*
Fonte: Elaborado pela autora (2021)

Note, também, que duas funções são declaradas no código “setup()” e “loop()”. A IDE do Arduino procura essas funções no momento de compilação e precisa que elas estejam sempre presentes, independentemente do código. A função “setup()” é executada uma única vez no microcontrolador, enquanto a função “loop()”, como o nome diz, será executada constantemente até que o dispositivo seja desligado.

Resumindo...

Sintaxe	Definição
<code>/* */</code>	Comentário longo no código, sendo comentários de uma linha <code>/* */</code> .
<code>setup()</code>	Função obrigatória que será executada uma única vez.
<code>loop()</code>	Função obrigatória que será executada constantemente.
<code>pinMode()</code>	Função nativa para definição de um pino como entrada ou saída.
<code>digitalWrite()</code>	Função nativa para definição do sinal digital a ser enviado.
<code>delay()</code>	Função nativa que atrasa o código em milissegundos.

Quadro 4 – Funções do código *Blink*

Fonte: Elaborado pela autora (2021)

Agora que você já sabe como executar códigos em dispositivos, podemos seguir para o código final do nosso ESP8266. Para isso, usaremos o código desenvolvido por Stefan Kremser (*spacehuhn*). Baixe o código disponibilizado em seu Github (https://github.com/spacehuhn/wifi_keylogger) e abra o arquivo “esp8266_saveSerial.ino”, encontrado na pasta “esp8266_saveSerial”. Veja que duas linhas definem um *SSID* e uma senha para um *Access Point Wi-Fi* na Figura abaixo. Você pode alterar para o nome e a senha que desejar.

```

esp8266_saveSerial | Arduino 1.8.13
File Edit Sketch Tools Help
esp8266_saveSerial
#include <ESP8266WiFi.h>
#include <FS.h>
#include <ESP8266mDNS.h>
#include <ESPAsyncTCP.h>
#include <ESPAsyncWebServer.h>
#include <SPIFFSEditor.h>
#include <EEPROM.h>

#define BAUD_RATE 115200

/* ===== CHANGE WIFI CREDENTIALS ===== */
const char *ssid = "definitely not a keylogger";
const char *password = "!keylogger"; //min 8 chars
/* ===== */

AsyncWebServer server(80);
FSInfo fs_info;
File f;

void setup() {
  Serial.begin(BAUD_RATE);

  //Serial.println(WiFi.SSID());
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);

```

Figura 36 – Código esp8266_saveSerial

Fonte: Elaborada pela autora (2021)

Conecte o dispositivo à porta USB e verifique se as configurações em “*Tools*” continuam como definimos. Selecione a porta desejada e clique em “*Verify*”, seguido de “*Upload*” até receber “*Done Compiling*” pela segunda vez. Pronto! Seu ESP8266 está programado, aguardando para poder ser o servidor *Web* do nosso *Hardware Keylogger*.

Agora, desconecte o ESP8266 da USB de seu computador e conecte o Arduino escolhido. Faça as modificações na aba “*Tools*” para atender essa nova placa e defina a porta USB do novo dispositivo. Abra o código “*keylogger.ino*”, encontrado na pasta “*keylogger*” e faça o *upload* para sua placa!

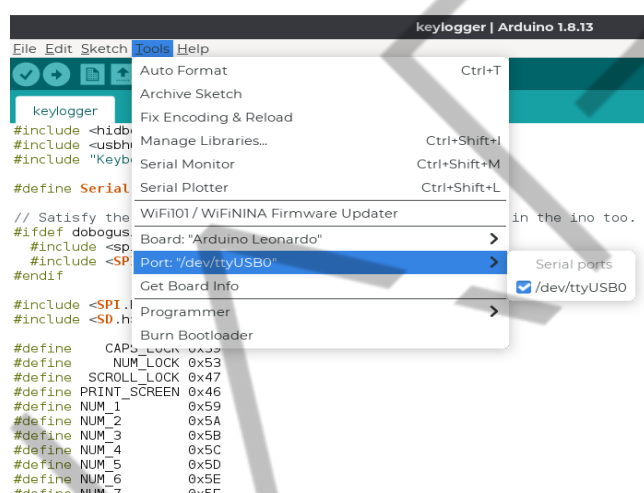


Figura 37 – Subindo o código keylogger para o Arduino Leonardo
Fonte: Elaborada pela autora (2021)

Lembra-se dos *jumpers* ligados aos pinos TX e RX que você removeu no início? Reconecte-os novamente, como mostrado na Figura “*Jumpers ESP8266 para Arduino Pro Micro*”. Se tudo ocorreu de forma correta, cheque os *Access Points* que aparecem para você em seu celular ou computador: em instantes deve aparecer um com o nome definido pela variável *SSID*.

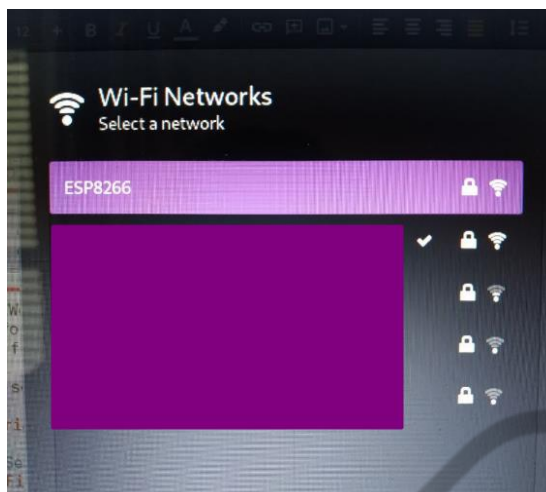


Figura 38 – ESP8266 Access Point
Fonte: Elaborada pela autora (2021)

No meu caso, coloquei o SSID como “ESP8266”. Conecte-se ao *Access Point* com a senha que definiu no código e navegue até a página <http://192.168.4.1>. Conecte um teclado na entrada USB do *USB Host Shield*. Como eu não possuo um no momento, programei um *BadUSB* para mandar as teclas simulando um teclado, como na Figura “Dispositivos providos do microcontrolador ATmega32u4”.

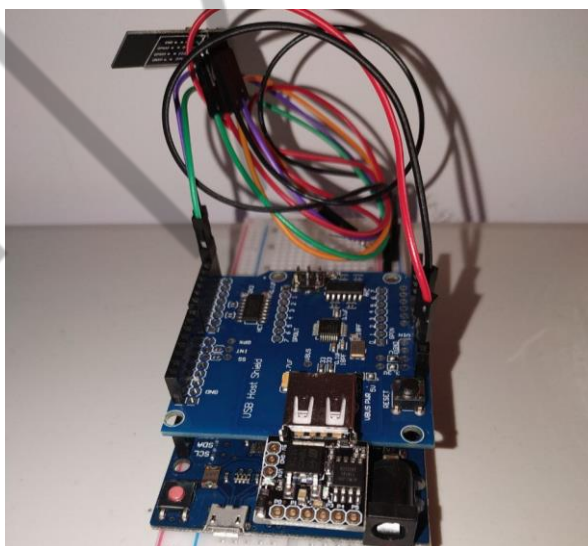


Figura 39 – BadUSB simulando um teclado
Fonte: Elaborada pela autora (2021)

Ao acessar a página, vemos as teclas sendo digitadas em tempo real! Para “limpar” as teclas recebidas, acesse <<http://192.168.4.1/clear>>.

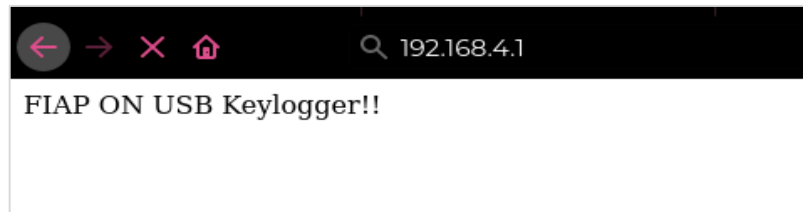


Figura 40 – Teclas enviadas em tempo real para a página Web, via ESP8266
Fonte: Elaborada pela autora (2021)

Uau! Com dispositivos acessíveis, como o Arduino, conseguimos criar uma ferramenta que, em tempo real, manda as teclas digitadas por um usuário para um ponto de acesso do nosso controle! Esse *gadget* já pode ser guardado na sua mala *hacker* para ser usado em seu próximo Pentest Físico como *Red Teamer*! O que mais podemos fazer? Já vimos que podemos simular facilmente um teclado! Que tipos de ataques podem ser feitos com um simples teclado? Veremos tudo isso e mais um pouco nas seções seguintes! Preparado? Bora lá!

5 BADUSB

O *BadUSB* é um ataque que explora uma vulnerabilidade inerente ao firmware USB. Esse ataque reprograma um dispositivo USB comum, fazendo com que ele atue como um dispositivo de HID. Uma vez alterado, o dispositivo USB é usado para executar comandos discretamente no computador das vítimas alvejadas.

O *exploit BadUSB* foi descoberto e exposto pela primeira vez pelos pesquisadores de segurança Karsten Nohl e Jakob Lell na conferência Black Hat de 2014.

Entretanto, não são todos os chips que apresentam essa vulnerabilidade específica. Pensando nisso, a *Hak5* desenvolveu um dispositivo em 2005 que já vinha com um *firmware* feito para funcionar como um dispositivo HID e, assim, funcionar como um *BadUSB*.



Figura 41 – Rubber Ducky
Fonte: Elaborada pela autora (2021)

Como você pode ver na Figura “Rubber Ducky”, o valor não está muito acessível para os brasileiros. Então, que tal criarmos um com Arduino? Alguns Arduinos vêm com uma habilidade embutida de poder funcionar como um dispositivo HID. Esses Arduinos compartilham de uma mesma coisa em comum: o microcontrolador ATmega32u4, que permite essa funcionalidade. Por isso, pegue o seu Arduino provido de um ATmega32u4 e mãos à obra!

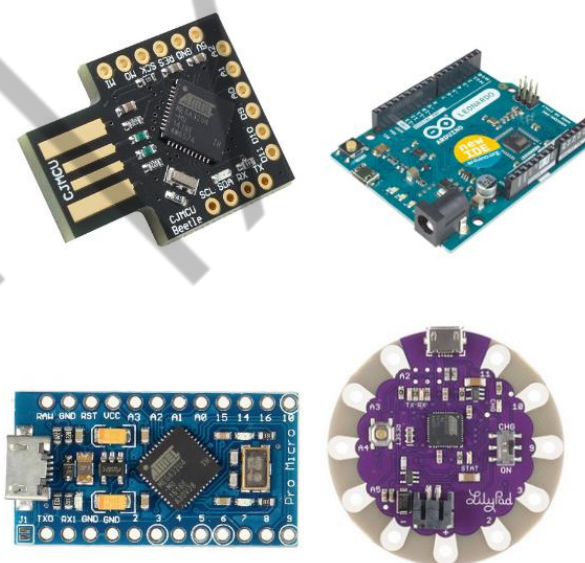


Figura 42 – Dispositivos providos do microcontrolador ATmega32u4
Fonte: Elaborada pela autora (2021)

5.1 Objetivo

Pensando em onde queremos chegar: **enviar teclas pré-programadas ao computador da vítima**, qual caminho lógico temos de percorrer?

1. Determinar o que é necessário para um dispositivo ser reconhecido como um teclado pelo computador: **Ser reconhecido como um dispositivo HID.**
2. Encontrar um CI que tenha essa capacidade: **ATmega32u4.**
3. Programar o CI: **Arduino IDE.**
4. *Hack the planet!*

5.2 Circuito

Para podermos usar o ATmega32u4 em sua capacidade HID, essencialmente precisamos que ele tenha as conexões elétricas da Figura “Conectando USB ao ATmega32u4”.

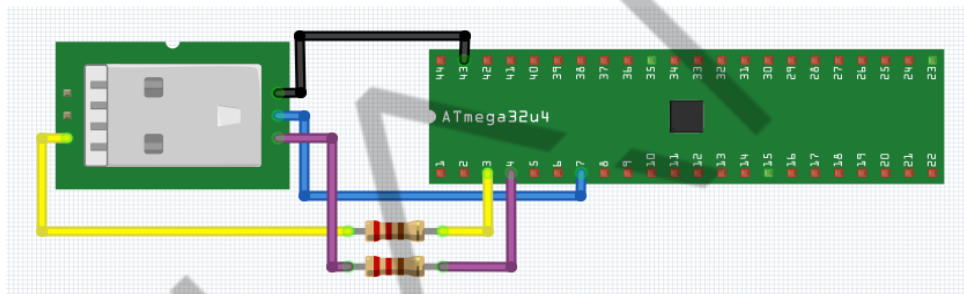


Figura 43 – Conectando USB ao ATmega32u4
Fonte: Elaborada pela autora (2021)

No “mundo real”, a Figura “Conectando USB ao ATmega32u4” ficaria parecida com a figura a seguir, ou seja, apenas uma conexão direta com a saída USB será suficiente para fazermos nosso ataque HID.



Figura 44 – Attiny85 conectado diretamente à saída USB
Fonte: Elaborada pela autora (2021)

Caso você não tenha adquirido algum *breakout board*, mas sim uma placa de desenvolvimento, provavelmente já terá essas conexões. Um *breakout board* é uma placa de componente único que, devido às suas dimensões muito pequenas,

dificultam a prototipagem rápida. Por isso, os pinos pequenos desses circuitos são expandidos de modo a se ter acesso a seus terminais, como na Figura “ATmega32u4 breakout board”.

Uma placa de desenvolvimento, em compensação, pode vir com alguns componentes a mais para auxiliar em seu desenvolvimento rápido, como um conversor FTDI para acesso via USB ou capacitores para estabilizar pontos importantes na placa. O Arduino, como você pode imaginar, é uma placa de desenvolvimento! Como podemos garantir que ele já possui as conexões necessárias para o ATmega32u4 se comunicar com uma saída USB?

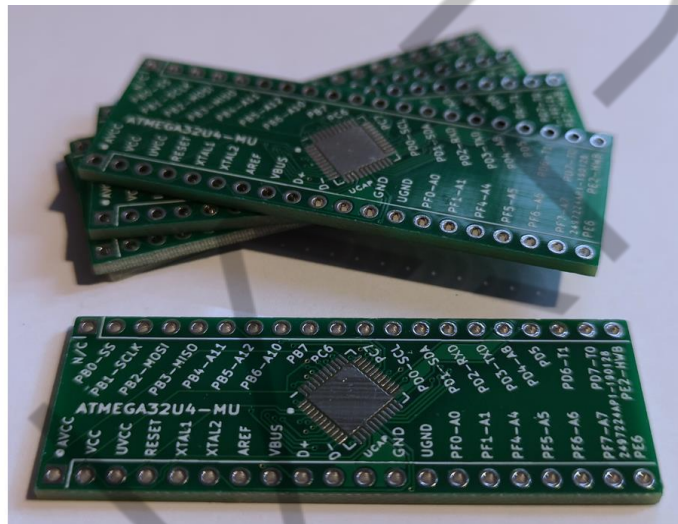


Figura 45 – ATmega32u4 breakout board
Fonte: Tindie (2021)

A melhor forma de ter certeza é procurar um arquivo chamado “esquemático” ou *schematics*, em inglês. Esse arquivo pode ser encontrado muitas vezes no próprio *datasheet* do dispositivo ou no site do fornecedor e ele apresenta um desenho gráfico das conexões elétricas da placa. Na Figura “Esquemático Arduino Pro Micro”, por exemplo, podemos ver exatamente onde é feita a interação com a porta USB no Arduino Pro Micro.

Comando	Descrição
DELAY	Promove uma pausa em milissegundos.
GUI	Emula a tecla Windows.
STRING	Escreve os caracteres definidos.
ENTER	Emula a tecla ENTER.

Quadro 5 – Definição de comandos da sintaxe *DuckyScript*
 Fonte: Elaborado pela autora (2021)

Bem simples, não é? Bom, como não pretendemos usar um *Rubber Ducky*, devemos portar esse código em linguagem *Duckyscript* para a linguagem que o Arduino fala: APL (*Arduino Programming Language*), uma linguagem baseada em C++. Graças à comunidade *Open-Source*, hoje temos uma forma de conversão direta acessando: <<https://dukweeno.github.io/Duckuino/>>.

Coloque o código demonstrado no Código “Código para escrita no notepad com *DuckyScript*” e selecione para compilar em Arduino.

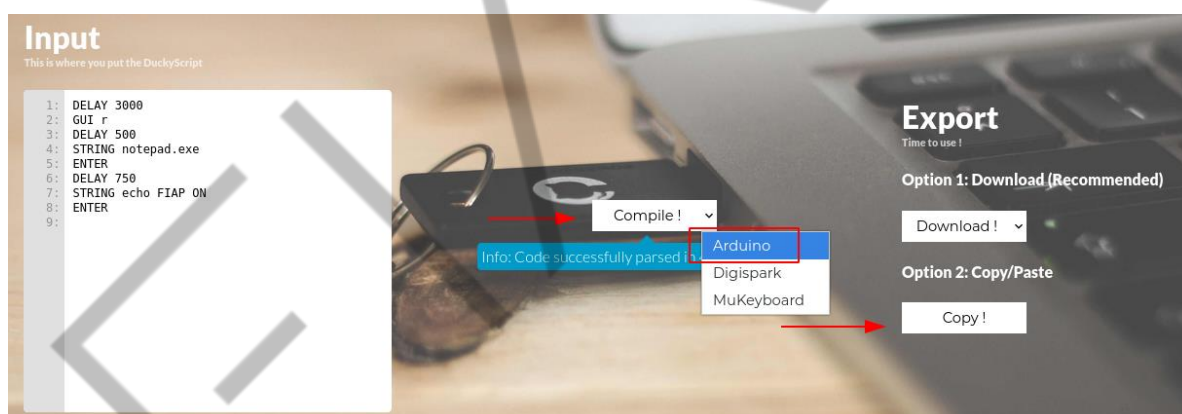


Figura 47 – Convertendo *DuckyScript* para Arduino
 Fonte: Elaborada pela autora (2021)

Cole como um novo sketch no seu Arduino IDE.



Figura 48 – Código convertido em Arduino

Fonte: Elaborada pela autora (2021)

Não esqueça de selecionar a sua placa e a porta USB corretamente no IDE.

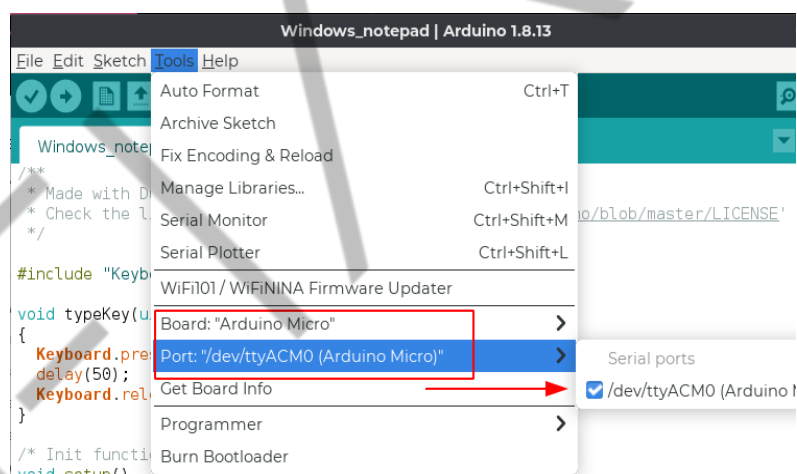


Figura 49 – Selecionando a placa e a porta corretas

Fonte: Elaborada pela autora (2021)

Clique em “Verify”, para garantir que não houve erros na colagem, seguido de “Upload”.

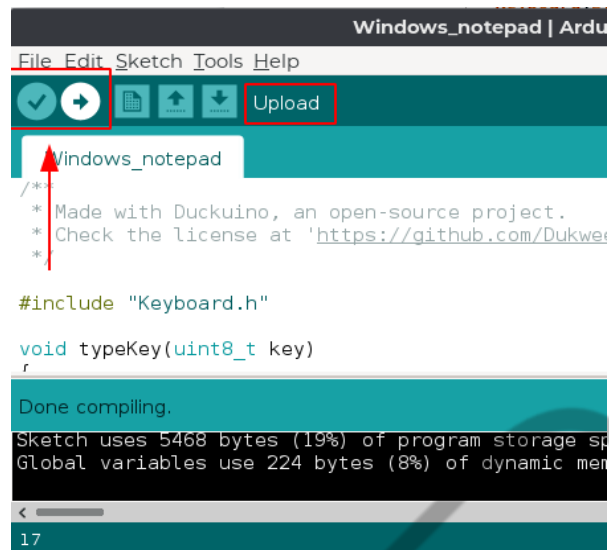


Figura 50 – Subindo o código para a placa
Fonte: Elaborada pela autora (2021)

Se tudo ocorreu de maneira correta, você agora tem um badUSB que abre o notepad em uma máquina Windows e escreve “FIAP ON”, como texto.

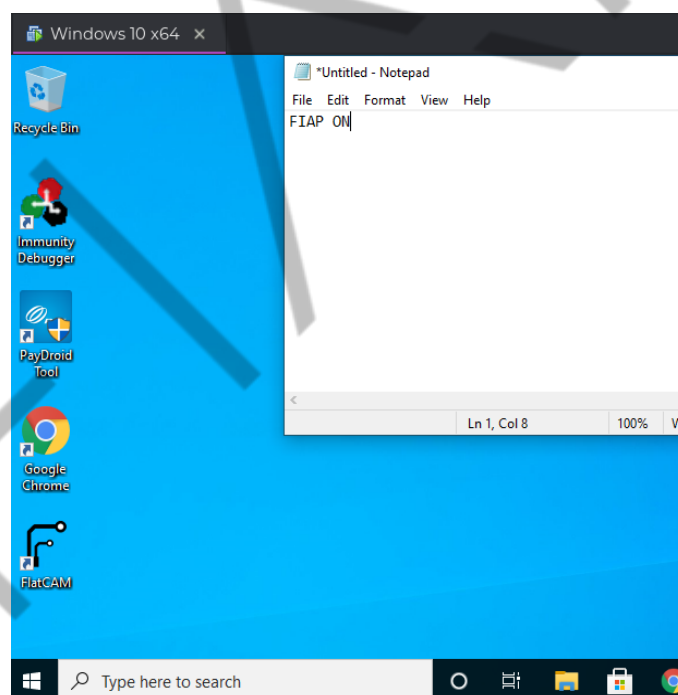


Figura 51 – Resultado do ataque
Fonte: Elaborada pela autora (2021)

O que pode ser feito com os comandos do *Duckyscript* variam da criatividade e malícia do Red Teamer. O vídeo “Reverse Shell - \$3 Arduino BadUSB” demonstra como chamar uma reverse shell em minutos com um Arduino Pro Micro e pode ser acessado em: <<https://www.youtube.com/watch?v=1ZyNU-RmBIs>>.

6 MASTERKEY

6.1 Objetivo

“Magos da eletricidade” em treinamento, preparados para o próximo nível? Agora que já sabem como montar um *Hardware Keylogger Wi-Fi* e um *BadUSB*, por que não juntar os dois?

Pensando no nosso objetivo: **enviar teclas pré-programadas e capturar teclas digitadas, controlando essas atividades via Wi-Fi**, qual caminho lógico temos de percorrer? Vamos às perguntas essenciais. Dessa vez, tente chegar nas respostas sozinho ou vá respondendo conforme o tutorial da seção.

1. O que é necessário para um dispositivo receber teclas digitadas? Portar-se como o USB Host da conexão USB com o teclado.
2. O que é necessário para um dispositivo ser reconhecido como um teclado pelo computador? Ser reconhecido como um dispositivo HID pelo computador.
3. Qual CI tem a capacidade de receber teclas digitadas? MAX3421E.
4. Qual CI tem a capacidade de ser reconhecida como um teclado pelo computador? ATmega32u4.
5. Como é feita a programação desse circuito? Arduino IDE.
6. Hack the planet!

Você consegue perceber o que o *Hardware Keylogger Wi-Fi* e o *BadUSB* têm em comum? Ambos precisam **ser reconhecidos como um dispositivo HID pelo computador**. Isso permite que utilizemos o próprio *Hardware Keylogger* como um *BadUSB*, sem a necessidade de adicionar uma nova placa de desenvolvimento!

6.2 Circuito

Vamos voltar ao nosso *Hardware Keylogger Wi-Fi*. Tenha ele montado em mãos, seguindo estas conexões:

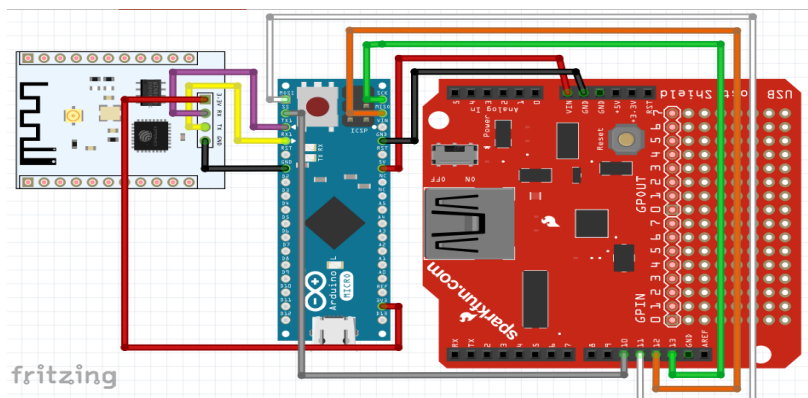


Figura 52 – Conexões do *Hardware Keylogger Wi-Fi*
Fonte: Elaborada pela autora (2021)

Caso você tenha optado pelo Arduino Leonardo em vez do Arduino Pro Micro, é esperado que você acabe com um circuito mais ou menos como o da figura a seguir.

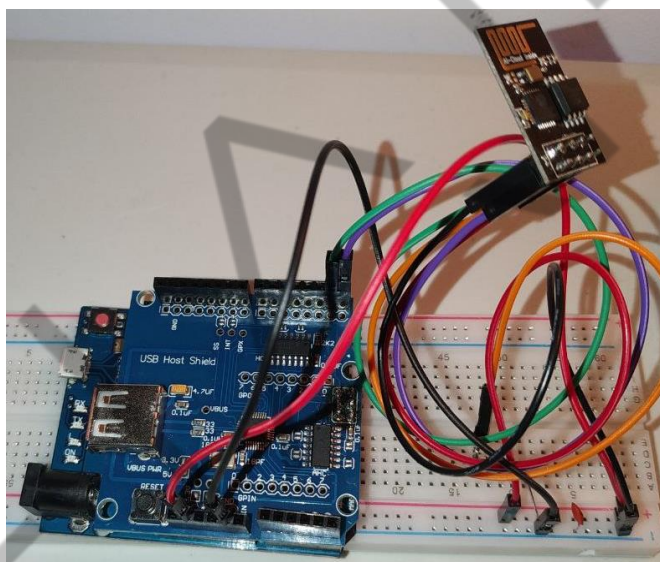


Figura 53 – Conexões do *Hardware Keylogger Wi-Fi* usando o Arduino Leonardo
Fonte: Elaborada pela autora (2021)

O USB Host Shield encaixa-se perfeitamente no Arduino Leonardo. Repare, entretanto, que existem alguns componentes não abordados anteriormente na conexão entre o Arduino e o ESP8266.

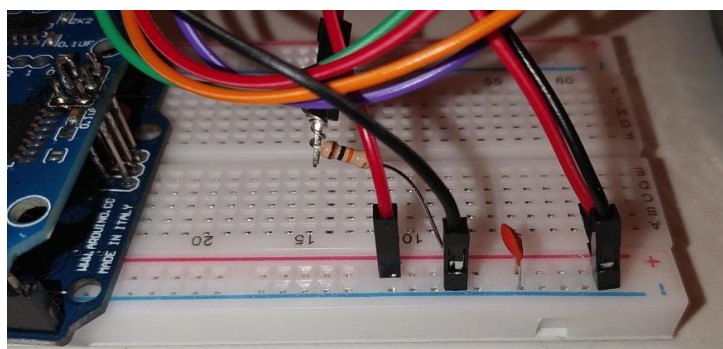


Figura 54 – Componentes presentes entre a conexão do Arduino Leonardo e o ESP8266

Fonte: Elaborada pela autora (2021)

Foi necessário o uso desses componentes devido à minha escolha do módulo Wi-Fi. Eu optei por uma placa de desenvolvimento ESP8266 mais simples, chamada ESP-01. Essa placa, por exemplo, não possui uma interface de comunicação direta para USB, exigindo um conversor serial.

No momento, não entraremos em detalhes do protocolo, mas saiba que se você comprou o módulo indicado na lista de materiais, então não terá de fazer modificações! Entretanto, vamos lembrar que o nosso objetivo principal neste capítulo é nos familiarizarmos com eletrônica e, assim, seguirmos para os próximos capítulos com mais experiência. Por isso, reflita um pouco sobre esses componentes. Você sabe identificar quais são e por que eles são importantes para essa conexão?

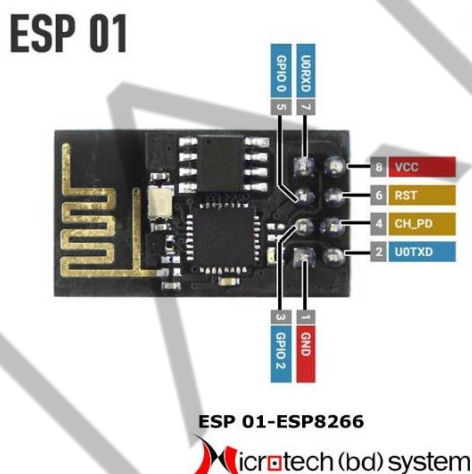


Figura 55 – Pinos do ESP-01
Fonte: Microtech (BD) Systems (2020)

Vamos começar conectando as linhas de energia VCC e GND. Ao analisarmos o *datasheet* do ESP8266, vemos que ele opera a 3.3V.

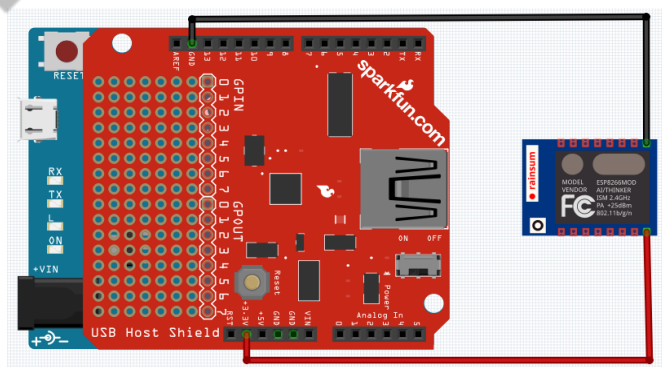


Figura 56 – Conexões de energia ESP-01
Fonte: Elaborada pela autora (2021)

Se analisarmos em mais detalhes o *datasheet* das duas placas de desenvolvimento, veremos que as correntes de operação do ESP8266 e a de fornecimento do Arduino Leonardo são diferentes. No caso, o ESP8266 opera com uma corrente maior (80mA) que o Arduino pode fornecer (50mA), como pode ser visto nas figuras a seguir.

Hardware	Peripheral Interface	UART/SDIO/SPI/I2C/I2S/IR Remote Control
		GPIO/ADC/PWM/LED Light & Button
	Operating Voltage	2.5 V ~ 3.6 V
	Operating Current	Average value: 80 mA
	Operating Temperature Range	-40 °C ~ 125 °C
	Package Size	QFN32-pin (5 mm x 5 mm)

Figura 57 – Corrente de operação no ESP8266
Fonte: Elaborada pela autora (2021)

Technical specs	
Microcontroller	ATmega32u4
Operating Voltage	5V
Input Voltage (Recommended)	7-12V
Input Voltage (limits)	6-20V
Digital I/O Pins	20
PWM Channels	7
Analog Input Channels	12
DC Current per I/O Pin	40 mA
DC Current for 3.3V Pin	50 mA
Flash Memory	32 KB (ATmega32u4) of which 4 KB used by bootloader
SRAM	2.5 KB (ATmega32u4)
EEPROM	1 KB (ATmega32u4)

Figura 58 – Corrente média de saída no pino 3.3V do Arduino Leonardo
Fonte: Elaborada pela autora (2021)

É pensando nisso que colocamos o capacitor.



Figura 59 – Capacitor cerâmico
Fonte: HuInfinito (2020)

O capacitor serve para muitas coisas, mas ele atua em sua maior parte como um reservatório. Vejamos o nosso caso: por mais que possamos atingir uma tensão suficiente para o ESP8266 (3.3V), limitamos a corrente a um valor de 50mA, ou seja, 30mA menor que a corrente de operação normal. E, afinal, qual a diferença entre tensão e corrente?

Nome	Definição	Unidade	Hierarquia
Tensão	Diferença de potencial entre dois pontos num circuito elétrico.	volts (V)	Causa
Corrente	Fluxo ordenado de partículas portadoras de carga elétrica	amperes (A)	Efeito

Quadro 6 – Diferença entre tensão e corrente
Fonte: HuInfinito (2020)

Para visualizar, considere-se em uma corrida de carros. Os carros são as partículas. O movimento em si dos carros é a corrente, e a largura da pista em que os carros passam é a tensão. A elétrica determina que quanto mais tensão tiver, mais energia pode fluir, mesmo que a intensidade da corrente seja a mesma. Transferindo essa regra para a corrida, quanto maior a largura da pista (tensão), maior o fluxo de carros que poderão passar simultaneamente (corrente), mesmo que a velocidade dos carros seja a mesma (intensidade da corrente). Mas e quando precisamos enviar um fluxo de carros maior que o tamanho da pista?

Como o capacitor possui um “reservatório” de energia, sempre que houver excesso de demanda de corrente que a fonte de alimentação (Arduino Leonardo) não pode atender sem reduzir a tensão (Lei de Joule), o capacitor libera essa corrente a partir de sua própria “reserva”.

Quando não há demanda, ele rapidamente reabastece sua “reserva”. É assim que ele preenche a lacuna entre a oferta e a demanda transitória. Em outras palavras, ele estabiliza a tensão.

O outro “problema” que devemos resolver para, assim, operarmos o ESP8266 sem causarmos danos é a sequência de inicialização dos pinos. No *datasheet* é explicado que o pino CHIP_EN (pino 7) deve receber energia no mesmo momento ou depois que o sistema todo é energizado. Para evitar que ele seja iniciado antes, é usado um **atraso na constante de tempo resistivo-capacitivo (RC)**, ou seja, é inserido um circuito composto de um resistor e um capacitor para atrasar minimamente o tempo que a corrente leva para chegar ao objetivo. Se precisar fazer cálculos,

seguimos a própria recomendação do datasheet e inserimos um resistor de 1k Ohm seguido de um capacitor de 100nF.

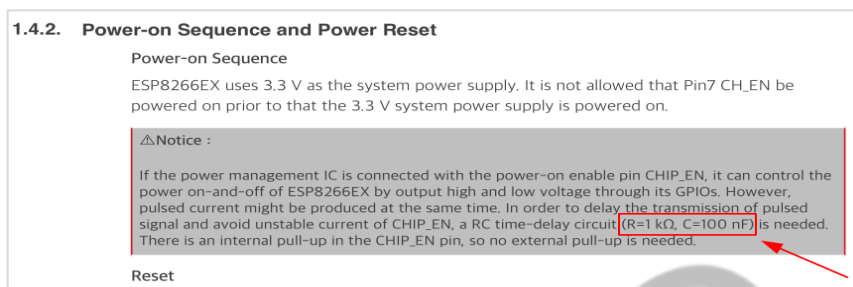


Figura 60 – Sequência de inicialização ESP8266
Fonte: Elaborada pela autora (2021)

Não se preocupe com a complexidade desses conceitos, compreenda apenas que, por mais complexo e inicialmente estranho que um circuito pareça, todas as respostas estão no melhor amigo dos *Hardware Hackers*: o *datasheet*.

Para finalizarmos as conexões do ESP-01, agora sabemos que ele precisa de algumas “necessidades especiais”, compostas por um estabilizador de tensão e um circuito de atraso resistivo-capacitivo. Vamos adicioná-los.

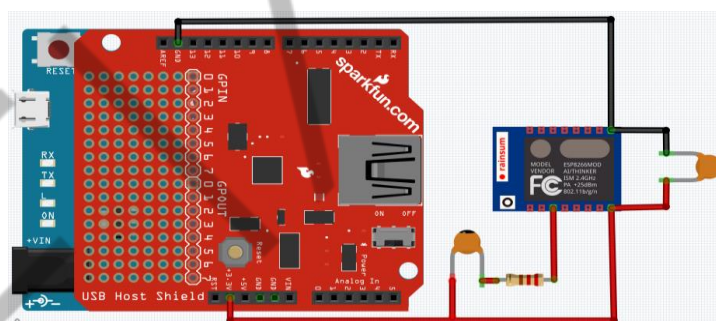


Figura 61 – Circuito com modificações exigidas no datasheet
Fonte: Elaborada pela autora (2021)

Precisamos apenas permitir que os módulos se comuniquem, agora, via UART. Para isso, conectamos os pinos “RX” e “TX” invertidos.

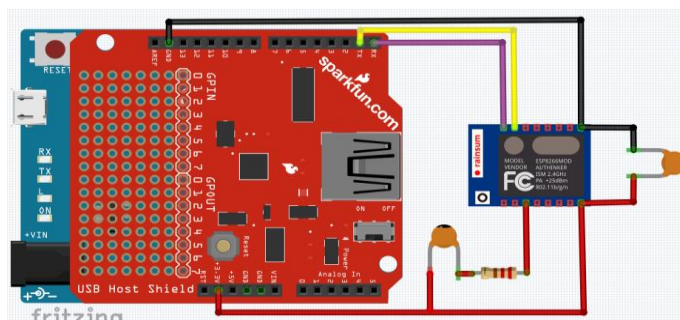


Figura 62 – Conexões de comunicação UART do ESP-01 ao Arduino Leonardo
Fonte: Elaborada pela autora (2021)

6.3 Código

Justcallmekoko teve a mesma ideia de juntar os dois dispositivos, criando outro que chamou de “*Masterkey*”. Ele apenas embutiu os chips *ATmega32u4*, *MAX3421E* e *ESP8266* em uma só placa de circuito! O código que ele criou para o *Masterkey* será o que usaremos para dar vida ao nosso *USB HID Keylogger Wi-Fi*.

Pegue o mesmo hardware que montou anteriormente e mãos à obra!



Figura 63 – Dispositivo Masterkey
Fonte: Elaborada pela autora (2021)

Primeiramente, baixe o código no repositório principal do Masterkey: <<https://github.com/justcallmekoko/USBKeylogger>>. Existem duas pastas que serão programadas separadamente, “esp8266_leonardo_keylogger.ino” (disponível em https://github.com/justcallmekoko/USBKeylogger/tree/master/esp8266_leonardo_keylogger) para subir o servidor Web no ESP8266, e “leonardo_usb_keylogger” (disponível em: https://github.com/justcallmekoko/USBKeylogger/tree/master/leonardo_usb_keylogger) para o Arduino Leonardo atuar como um USB *Host* e *HID* ao mesmo tempo.

Da mesma forma como foi programado o ESP8266, iremos programá-lo apenas subindo um código diferente. Siga os passos a seguir:

1. Desconectar os pinos TX e RX.
2. Faça o upload do código “esp8266_leonardo_keylogger.ino” para o seu ESP8266.
3. Faça o upload do código “leonardo_usb_keylogger.ino” para o seu Arduino de escolha.

4. Reconecte os pinos TX e RX, seguindo as conexões da figura “Jumpers ESP8266 para Arduino Pro Micro”.
5. Ligue o Hardware em uma fonte de energia.
6. Conecte-se ao Access Point “Masterkey” com a senha “masterkey”.
7. Navegue até a página <http://192.168.4.1/> e conecte um teclado USB à entrada USB do seu USB Host Shield.
8. Brinque com as possibilidades!

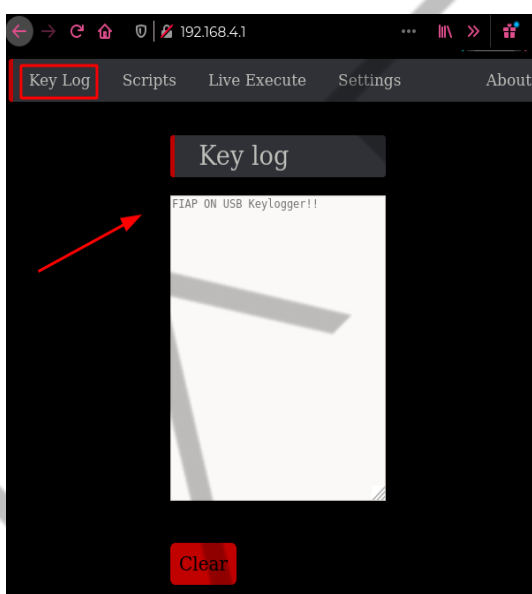


Figura 64 – Página Web Keylogger Wi-Fi Masterkey
Fonte: Elaborada pela autora (2021)

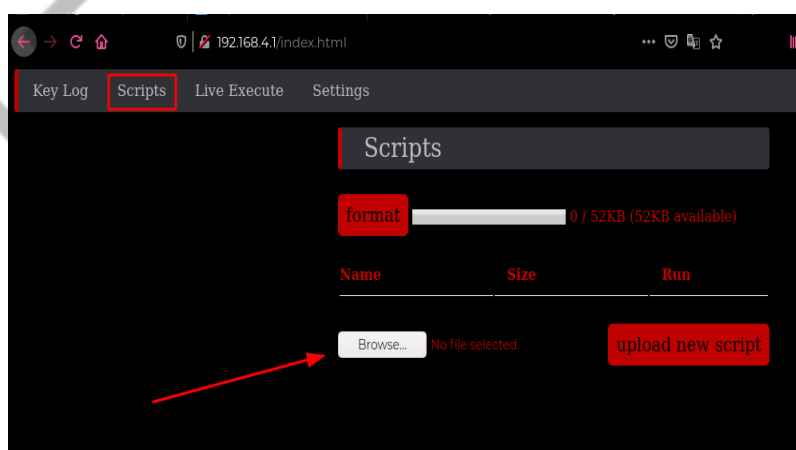


Figura 65 – Aba de execução de arquivos Duckyscript
Fonte: Elaborada pela autora (2021)

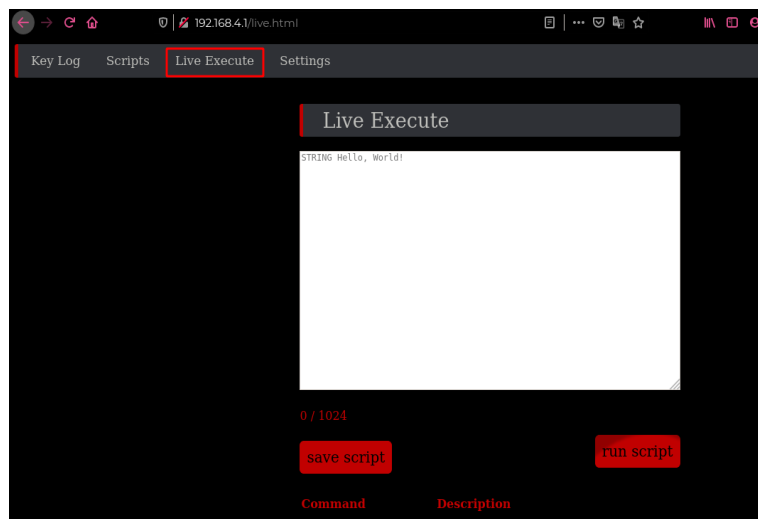


Figura 66 – Aba de execução *DuckyScript* em tempo real
Fonte: Elaborada pela autora (2021)

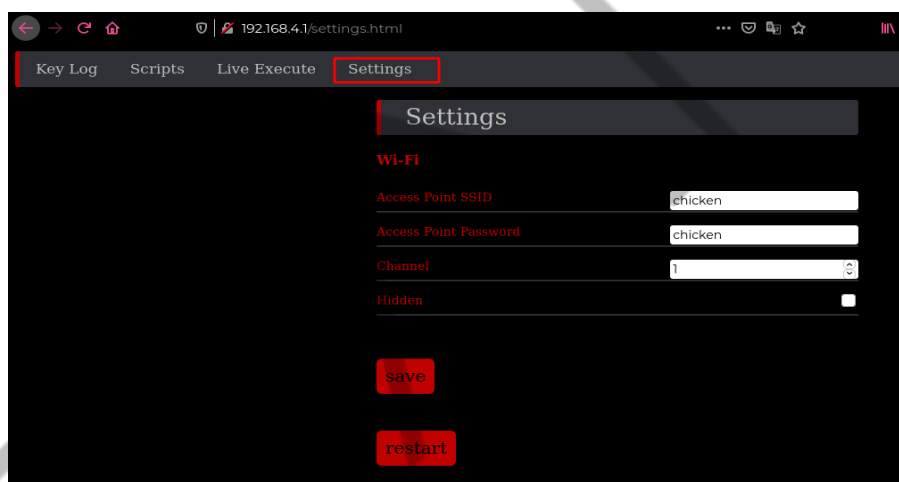


Figura 67 – Aba de configurações *Masterkey*
Fonte: Elaborada pela autora (2021)

Neste capítulo, então, criamos três dispositivos diferentes e que podem ser incorporados no dia a dia de um *Red Teamer*! Lembre-se de sempre usá-los com autorização de todas as partes previamente. Espero que tenham se divertido e nos veremos no próximo capítulo!

REFERÊNCIAS

ARDUINO. **Arduino Micro**. 2023. Disponível em: <<https://store-usa.arduino.cc/products/arduino-micro?selectedStore=us>>. Acesso em: 22 set. 2023.

ATMEL. **ATmega16U4/ATmega32U4**. 2017. Disponível em: <https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7766-8-bit-AVR-ATmega16U4-32U4_Datasheet.pdf>. Acesso em: 29 jun. 2021.

ESPRESSIF SYSTEMS. **ESP8266EX DATASHEET**. 2020. Disponível em: <https://www.espressif.com/sites/default/files/documentation/0a-esp8266ex_datasheet_en.pdf>. Acesso em: 2 set. 2021.

FIAP SCHOOL. **A FIAP School em Fotos**. [s.d.]. Disponível em: <<https://www.fiap.com.br/colegio/fotos/>>. Acesso em: 29 jun. 2021.

GITHUB. **Brandonlw/Psychson**. [s.d.]. Disponível em: <<https://github.com/brandonlw/Psychson>>. Acesso em: 29 jun. 2021.

GITHUB. **justcallmekoko/USBKeylogger**. 2021a. Disponível em: <<https://github.com/justcallmekoko/USBKeylogger>>. Acesso em: 29 jun. 2021.

GITHUB. **Spacehuhn/Wi-Fi Keylogger**. 2021b. Disponível em: <https://github.com/spacehuhn/wifi_keylogger>. Acesso em: 29 jun. 2021.

HAK5. **Field Kits**. [s.d.]. Disponível em: <<https://shop.hak5.org/pages/field-kits>>. Acesso em: 29 jun. 2021.

MAXIM INTEGRATED. **MAX3421E USB Peripheral/Host Controller with SPI Interface - Maxim Integrated**. [s.d.]. Disponível em: <<https://www.maximintegrated.com/en/products/interface/controllers-expanders/MAX3421E.html>>. Acesso em: 29 jun. 2021.

MICHAELIS ON-LINE. **Hacker**. 2021. Disponível em: <<https://michaelis.uol.com.br/busca?r=0&f=0&t=0&palavra=hacker>>. Acesso em: 29 jun. 2021.

PLUG IN To Win - DIY Bad USB. **0x00SEC**, 2021. Disponível em: <<https://0x00sec.org/t/plug-in-to-win-diy-bad-usb-part-3-3/1159>>. Acesso em: 2 set. 2021.

REVERSE Shell - \$3 Arduino BadUSB. [S. l.: s. n.], 2016. 1 vídeo (5min 33s) Disponível em: <<https://www.youtube.com/watch?v=1ZyNU-RmBI8>>. Acesso em: 29 jun. 2021.

SPARK FUN ELETRONICS. **Pro Micro (Dev-12640)**. [s.d.]. Disponível em: <<https://cdn.sparkfun.com/datasheets/Dev/Arduino/Boards/ProMicro16MHzv1.pdf>>. Acesso em: 29 jun. 2021.

SPARK FUN ELETRONICS. **Pro Micro**. [s.d.]. Disponível em: <https://cdn.sparkfun.com/datasheets/Dev/Arduino/Boards/Pro_Micro_v13b.pdf>. Acesso em: 29 jun. 2021.

SPARK FUN ELETRONICS. **USB Peripheral/Host Controller with SPI Interface**. [s.d.]. Disponível em: <<https://www.sparkfun.com/datasheets/DevTools/Arduino/MAX3421E.pdf>>. Acesso em: 29 jun. 2021.

TONYLABS. **ARDUINO UNO & ATMEGA 32U4 PINOUT DIAGRAM**. 2016. Disponível em: <<http://www.14core.com/wp-content/uploads/2015/06/atmel-atmega32u4-pinout-diagram.png>>. Acesso em: 29 jun. 2021.

USB IMPLEMENTERS FORUM. **Document Library**. 2021. Disponível em: <<https://www.usb.org/documents>>. Acesso em: 29 jun. 2021.